Advanced Data Structures

Chapter 2

Instructor: Shailesh B. Pandey
Everest Engineering College

B-Tree Motivation

- We assume that everything in a (search) tree is kept within the main memory
- What if the data items contained in a search tree do not fit into the main memory?
- Example, searching the National ID database (assume 8 bytes per citizen is stored)
  - Population of Nepal: 29,164,578 (as per the 2021 census) ≈ 233 MB

Cycles to Access Storage

Storage type	**Access type**	**Number of cycles**
CPU registers	Random	1
L1 cache	Random	2
L2 cache	Random	30
Main memory	Random	2.5×10^2
Hard disk	Random	3×10^7
	Streamline	5×10^3

Search trees on disks

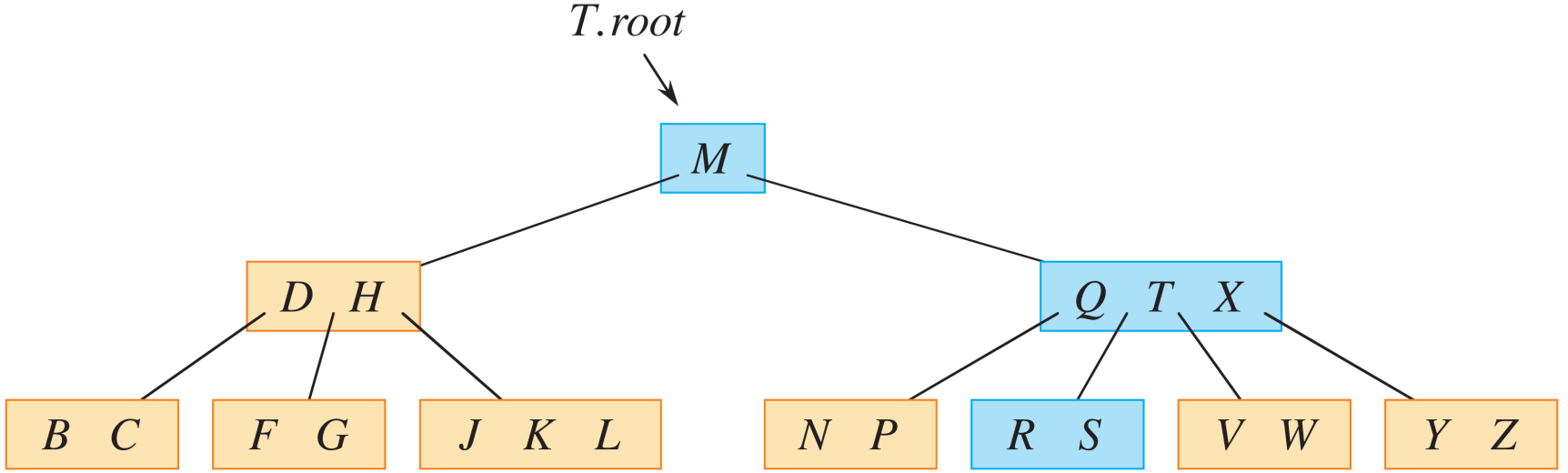
- A majority of the tree operations (search, insert, delete, etc.) will require O(lg n) disk accesses where n is the number of data items in the search tree

- The main challenge is to reduce the number of disk accesses for processing per data item

- An m-ary search tree allows m-way branching. As branching increases, the depth decreases. A complete binary tree has a height of [lg n] but a complete m-ary tree has a height of $[\log_m n]$.

B-Tree

- B-Tree is a low-depth self-balancing tree. The height of a B-Tree is kept low by putting maximum possible keys in a B-Tree node.



- The B-Trees have a higher branching factor (also termed as the order) to reduce the depth.

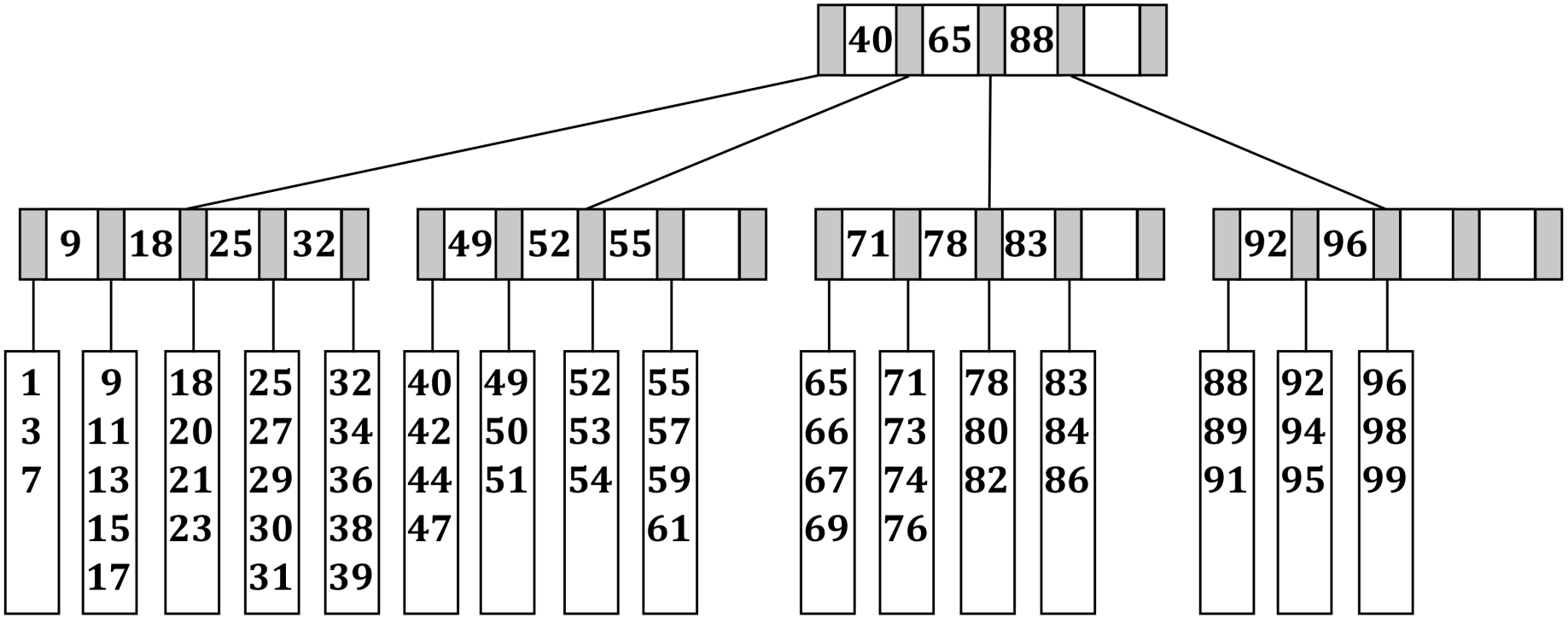
Definition

A B-Tree of order m is an m-ary tree with the following properties:

- The data items are stored at leaves.
- The non-leaf nodes store up to m − 1 keys to guide the searching; The key i represents the smallest key in subtree i + 1.
- The root is either a leaf or has between 2 and m children.
- All non-leaf nodes (except the root) have between [m/2] and m children
- All leaves are at the same depth and have between [k/2] and k data items, for some k.

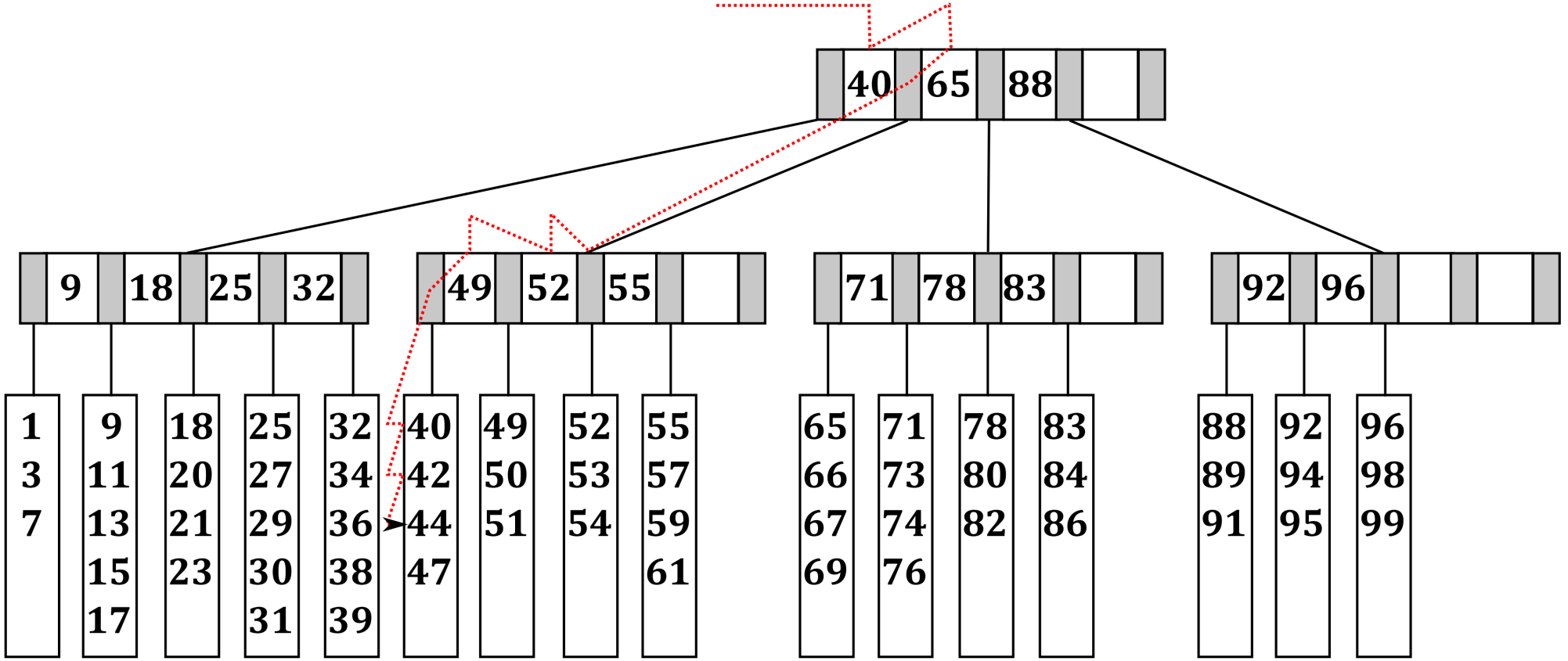
B-Tree

- Order 5
- Depth 3
- m=5
- k=5



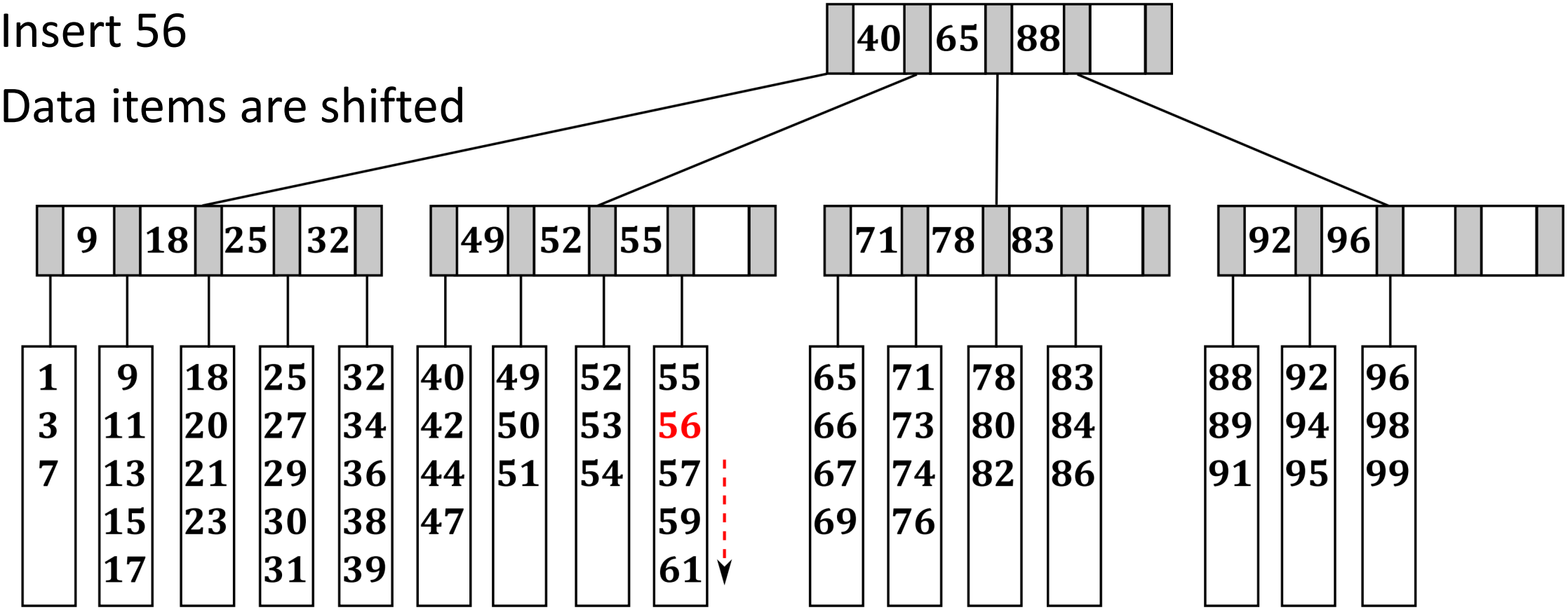
Searching

- Search 44



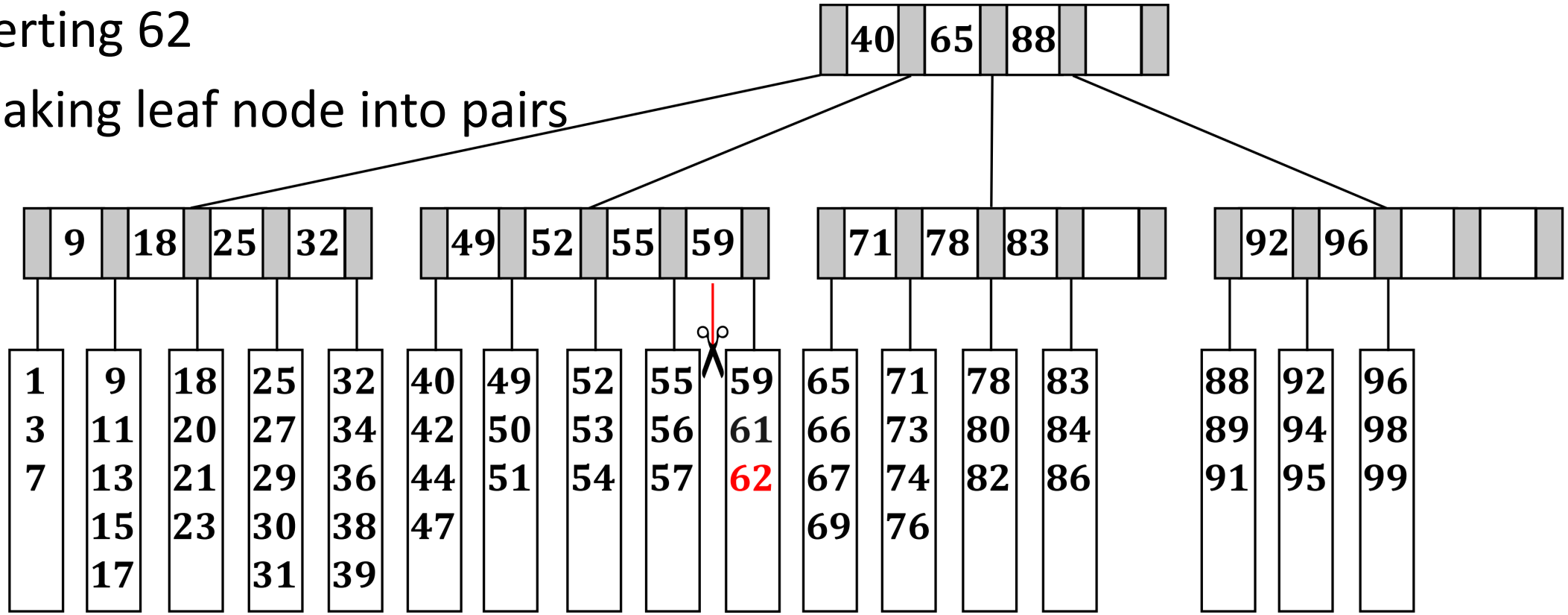
Inserting

- Insert 56
- Data items are shifted

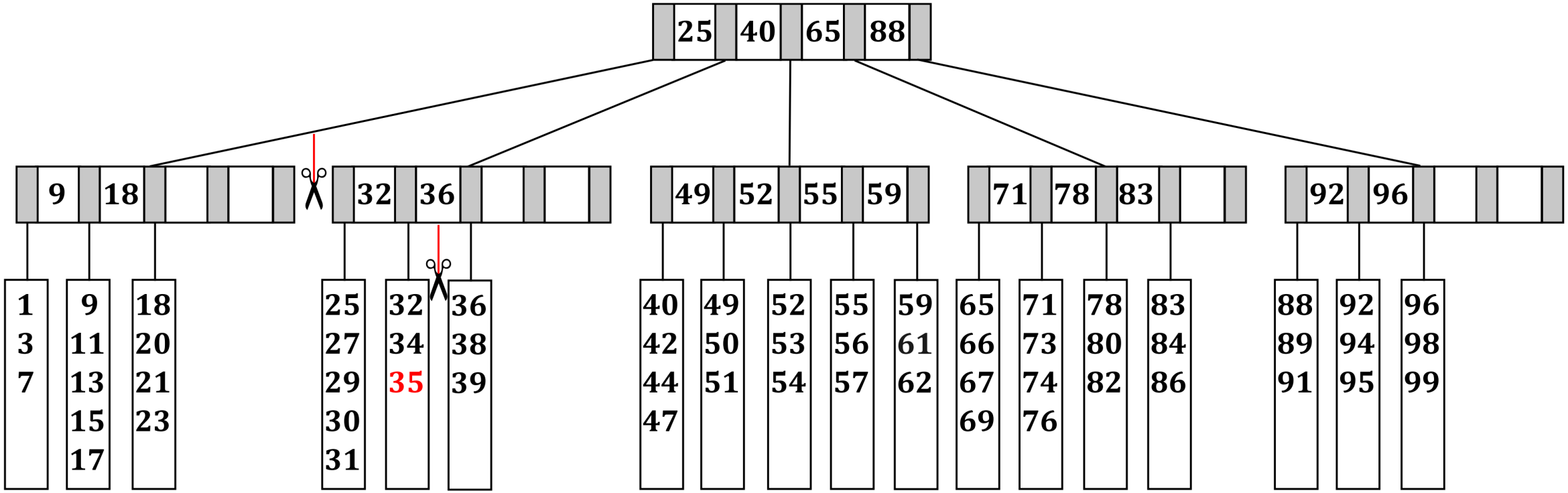


Inserting

- Inserting 62
- Breaking leaf node into pairs



Inserting



Inserting 35

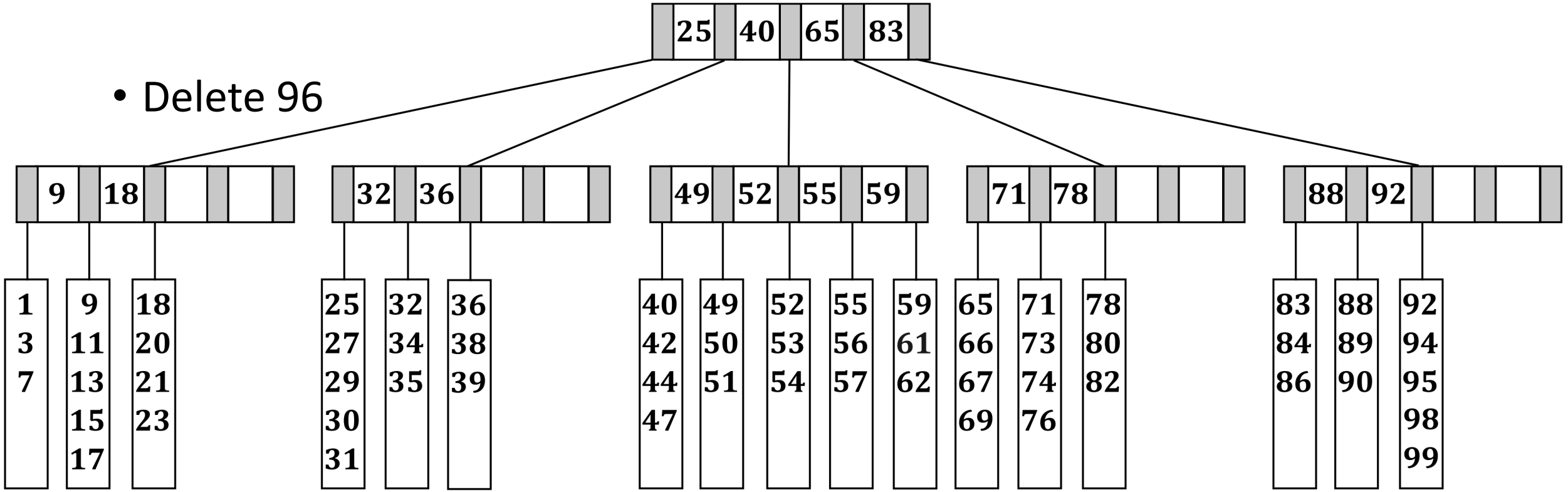
- Breaking into pairs and inclusion of a new non-leaf node

Insertion Algorithm

1. Insert the key in its leaf in sorted order
2. If the leaf ends up with L+1 items, “overflow”
   - Split into two nodes:
     - Old with [(L+1)/2] items and new with [(L+1)/2] items
   - Add the new child to the parent
   - If parent ends up with M+1 children, “Overflow”
3. If an internal node ends up with M+1 children, “Overflow”
   - Split into two nodes:
     - Old with [(M+1)/2] children and new with [(M+1)/2] children
   - Add the new child to the parent
   - If parent ends up with M+1 children, “Overflow”
4. Split an overflowed root in two. Hand new nodes under a new root
5. Propagate keys up the tree

Deleting

- Delete 96



Deletion requires merging of a leaf node with another node.

Deletion Algorithm

1. Remove the key from its leaf
2. If the leaf ends up with fewer than L/2 items, “underflow”
   - Adpot data from a neighbour, update the parent
   - If adopting won’t work, delete node and merge with neighbour
   - If parent ends up with fewer than M/2 children, “Underflow”
3. If an internal node ends up with fewer than M/2 children, “Underflow”
   - Adpot data from a neighbour, update the parent
   - If adopting won’t work, delete node and merge with neighbour
   - If parent ends up with fewer than M/2 children, “Underflow” Split an overflowed root in two. Hand new nodes under a new root
4. If the root ends up with only one child, make the child the new root of the tree
5. Propagate keys up through tree

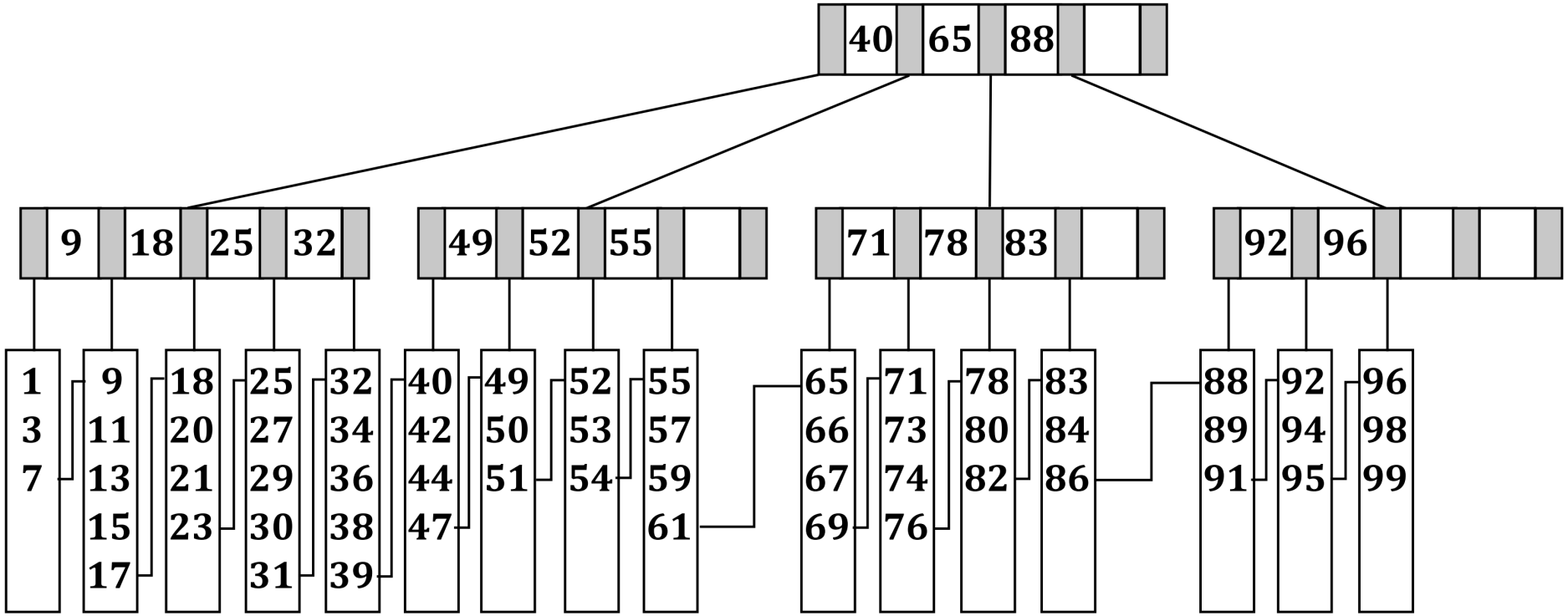
B^+-Trees

A B+-Tree of order m is an m-ary tree with the following properties:

- The data items are stored at leaves.
- The non-leaf nodes store up to m − 1 keys to guide the searching; The key i represents the smallest key in subtree i + 1.
- The root is either a leaf or has between 2 and m children.
- All leaves are at the same depth and have up to k data items, for some k.

B$^+$-Tree

- A B+-Tree of order 5 and depth 3 that contains 59 data items.



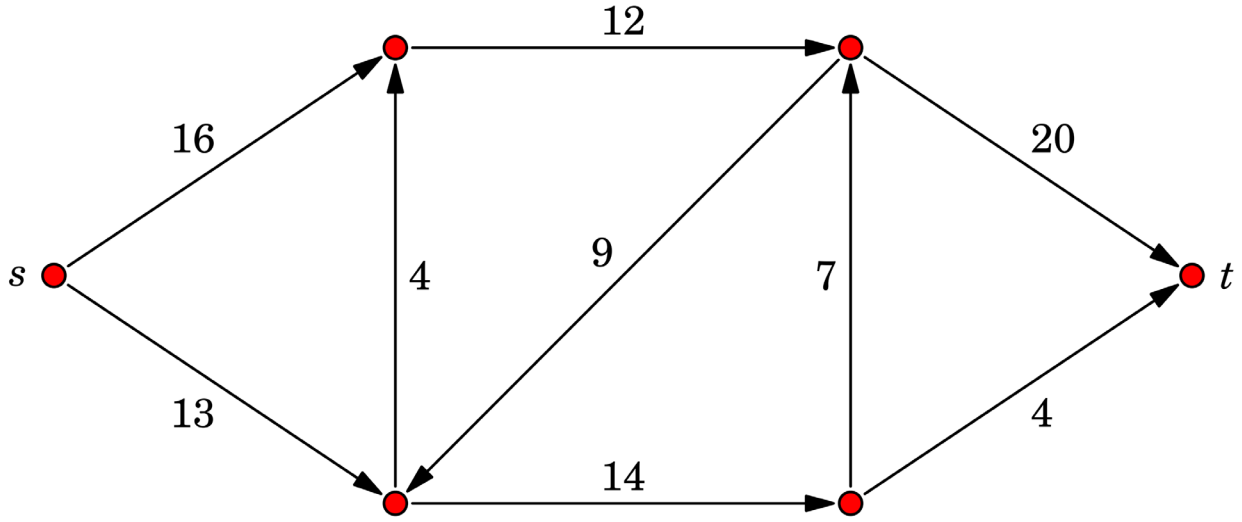
Network Flows

- When we concern ourselves with shortest paths, we interpret the edge weights of an undirected graph as distances.

- We will ask a question of a different sort. We start with a directed weighted graph G with two distinguished vertices s (the source) and t (the sink).

- We interpret the edges as unidirectional water pipes, with an edge's capacity indicated by its weight. The maximum flow problem then asks, how can one route as much water as possible from s to t?

Flow Network



- A flow network is a directed graph G = (V,E) with distinguished vertices s (the source) and t (the sink), in which each edge (u,v) ∈ E has a nonnegative capacity c(u,v).

- We require that E never contain both (u,v) and (v,u) for any pair of vertices u,v (so in particular, there are no loops). Also, if u,v ∈ V with (u,v) ∉ E, then we define c(u,v) to be zero.

Max Flow Problem

- Now, to define our problem, we have two constraints. The capacity constraint: each edge u, v in a flow network has some flow f(u, v) attached such $0 \leq f(u, v) \leq c(u, v)$.

- The flow conservation that the flow entering a node must be the same as the flow exiting a node ($\sum_{v \in V} f(u, v) = \sum_{v \in V} f(v, u)$).

- Then, in the maximum-flow problem, given G, s, t, and c we try to maximize the flow moving across the network (or the total flow moving out of the source or into the sink).

Ford Fulkerson

Ford-Fulkerson(G, S, T):
    initialize flow f(u, v) = 0 for all edges (u, v) in G

    while there exists an augmenting path P in the residual graph G_f:
        find the bottleneck capacity c_f(P) = min(c_f(u, v)) for all (u, v) in P
        for each edge (u, v) in P:
            f(u, v) = f(u, v) + c_f(P)   // Add flow
            f(v, u) = f(v, u) - c_f(P)   // Reverse flow

    return the total flow out of source S

Edmonds-Karp

```
Edmonds-Karp(G, S, T):
   initialize flow f(u, v) = 0 for all edges (u, v) in G

   while BFS(G_f, S, T) finds an augmenting path P:
      find bottleneck capacity c_f(P) = min(c_f(u, v)) for all (u, v) in P

      for each edge (u, v) in P:
         f(u, v) = f(u, v) + c_f(P)   // Add flow
         f(v, u) = f(v, u) - c_f(P)   // Update reverse flow

   return sum of flows on edges leaving the source S
```

Shortest Path

- Given a directed weighted graph G = (V,E,w) and two vertices x and y, find the least-cost path from x to y in G

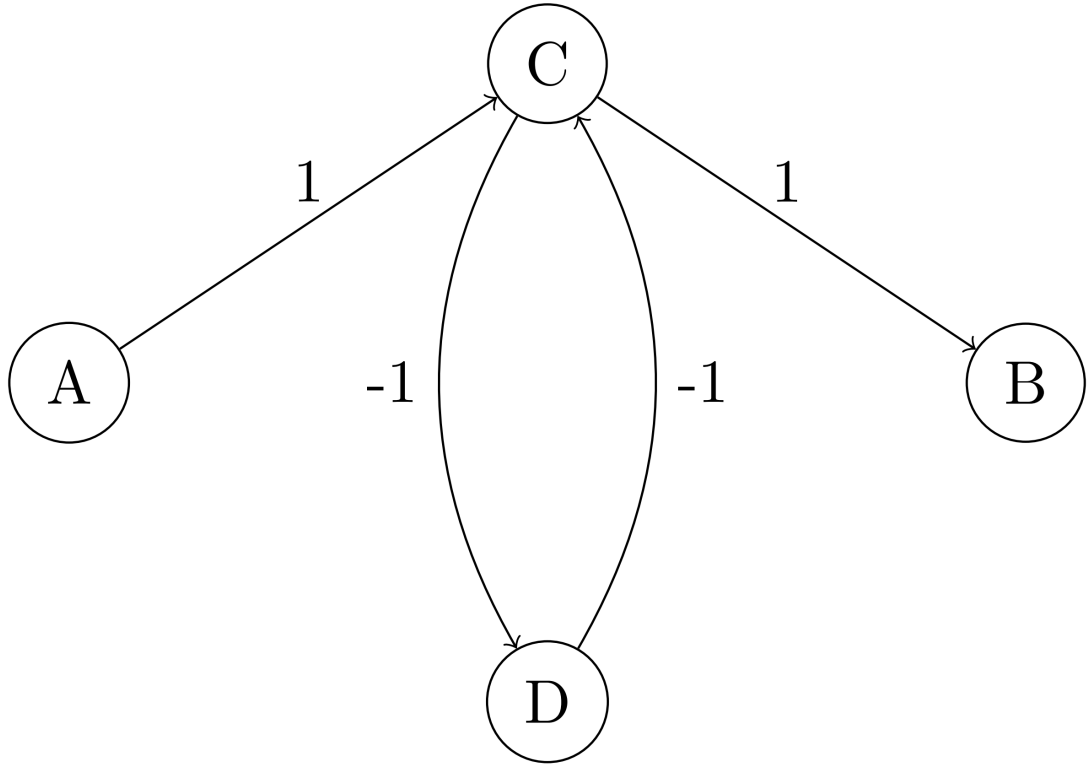
- The task is to find a path P that minimizes the function

$$w(P) = \sum_{u \to v \in P} w(u \to p)$$

- What if the weights are negative?

Negative Cycles

- What is the shortest path from A to B ?
  - Ans: There is no shortest path
- C and D from a negative cycle in this graph
  - sum of edges is negative



Dijkstra's Algorithm

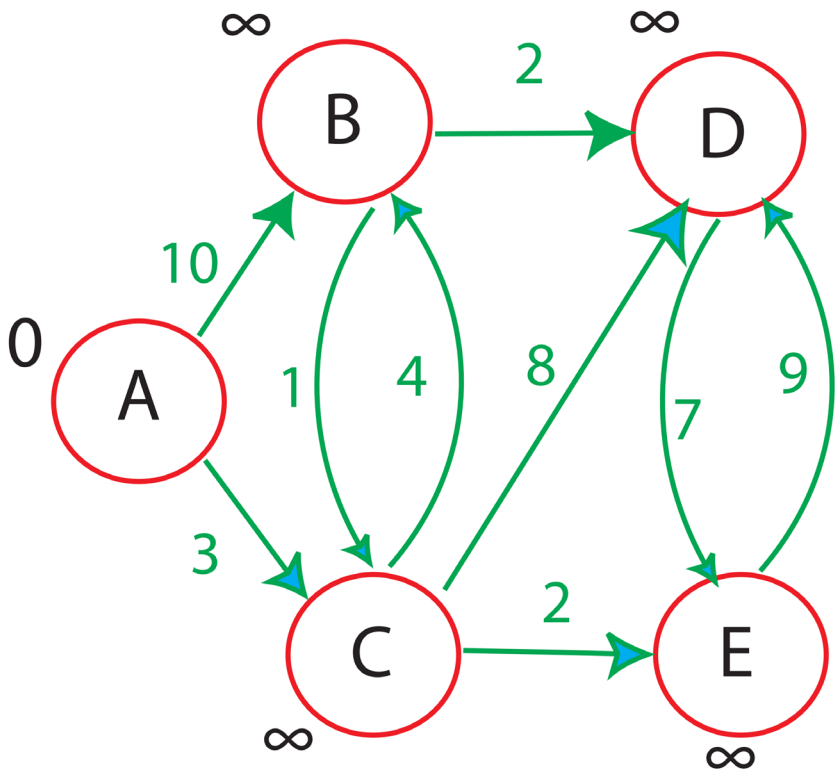
For each edge $(u, v) \in E$, assume $w(u, v) \geq 0$, maintain a set S of vertices whose final shortest path weights have been determined. Repeatedly select $u \in V - S$ with minimum shortest path estimate, add u to S, relax all edges out of u.

RELAX(u, v, w)

if d[v] > d[u] + w(u, v)

then d[v] = d[u] + w(u, v)

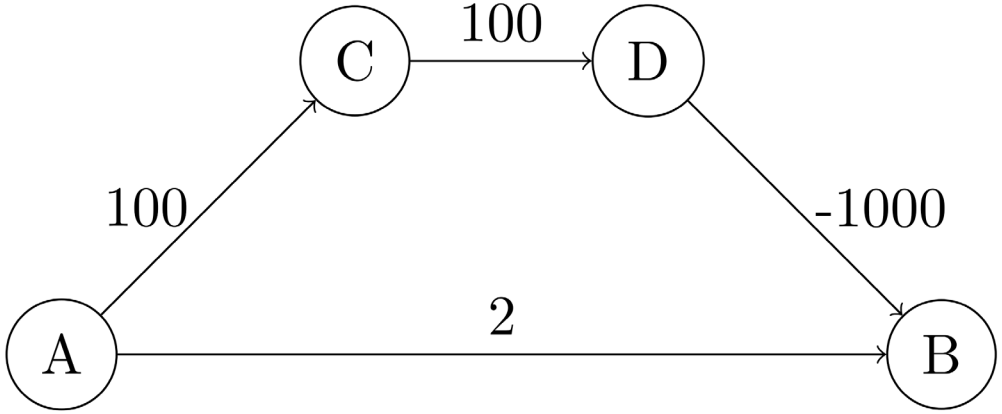
Dijkstra's Algorithm



S = { }	{ A B C D E } =	Q	
S = { A }	0 ∞ ∞ ∞ ∞		
S = { A, C }	0 10 3 ∞ ∞	←	after relaxing edges from A
S = { A, C }	0 7 3 11 5	←	after relaxing edges from C
S = { A, C, E }	0 7 3 11 5		
S = { A, C , E, B}	0 7 3 9 5	←	after relaxing edges from B

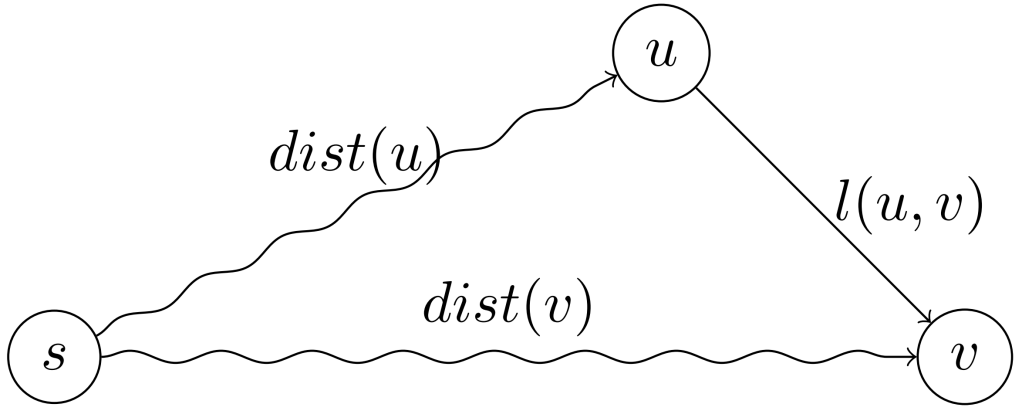
Bellman-Ford Algorithm

- The Bellman-Ford algorithm is a way to find single source shortest paths in a graph with negative edge weights



- Dijkstra's algorithm does not work if there is an edge in the graph with a negative weight
  - Why? Negative cycles.

Bellman-Ford



The Bellman-Ford algorithm utilizes the same sort of distance update operation/relaxation that Dijkstra's uses:

dist(v) = min(dist(v), dist(u) + l(u, v))

- Dijkstra has the privilege to stop performing this update operation for node v when it is popped from the priority queue since a shorter path to v would never happen in the future.

- However, with negative edges, Bellman-Ford may need to apply this update operation many more times to account for future shorter paths.

Pseudocode

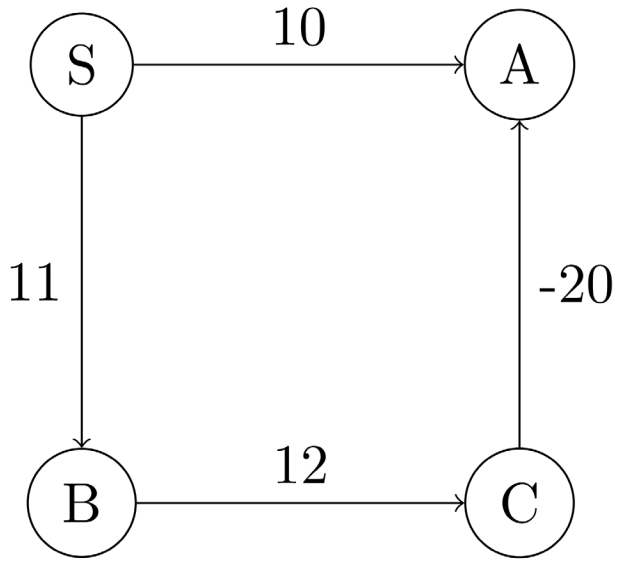
```
def bellmanFord(G, l, s):
        for all v in V:
                dist(u) = infinity

dist(s) = 0
for i=1...(|V| - 1)
        for each e=(u,v) in E
                dist(v) = min{dist(v), dist(u) + l(u, v)}
```

Example

- Distance value for each round

Round	dist(S)	dist(A)	dist(B)	dist(C)
0	0	∞	∞	∞
1	0	10	11	∞
2	0	10	11	23
3	0	3	11	23



The key observation here is the update on dist(A) in Round 3:

dist(A) = min(dist(A), dist(C) + l(C, A))

This would evaluate to min(10, 23 − 20) = 3

Question

- What is the worst-case time complexity of Bellman-Ford?

- Why do we need |V| - 1 iterations?

Johnson's Algorithm

- Johnson's all-pairs shortest path algorithm computes a cost for each vertex, so that the new weight of every edge is non-negative, and then computes shortest paths with respect to the new weights using Dijkstra's algorithm.

- First, suppose the input graph has a vertex s that can reach all the other vertices. Johnson's algorithm computes the shortest paths from s to the other vertices, using Bellman-Ford (which doesn't care if the edge weights are negative), and then reweights the graph using the cost function $\pi(v) = dist(s, v)$. The new weight of every edge is:

$w'(u \rightarrow v) = dist(s, u) + w(u \rightarrow v) - dist(s, v)$ [This weight is always positive]

Geometric data

- Classical dictionary data structure largely concerns with one-dimensional keys

- What to do when the keys are multi-dimensional?
  - Spatial maps, computer vision/graphics, machine learning etc.

- Queries
  - Nearest Neighbour Search: closest point to the query point
  - Range Searching: Give a rectangle or a circle find the points that line inside
  - Point Location: Determine the region of the subdivision containing this point

Multi-Dimensional Data

- There are similarities with 1-D data
  - Tree structure
  - Balance O(lg n)
  - Internal and leaf nodes

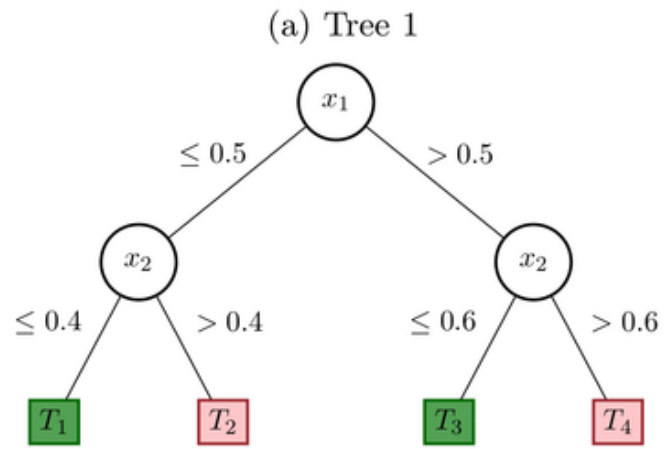
- Difference
  - No natural total order (no one unique way to sort the data e.g. x and y coordinates)
  - Tree rotation may not be meaningful

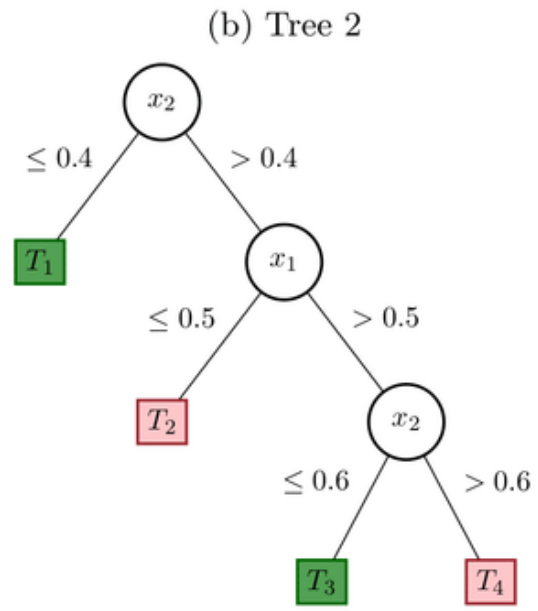
Data Structure

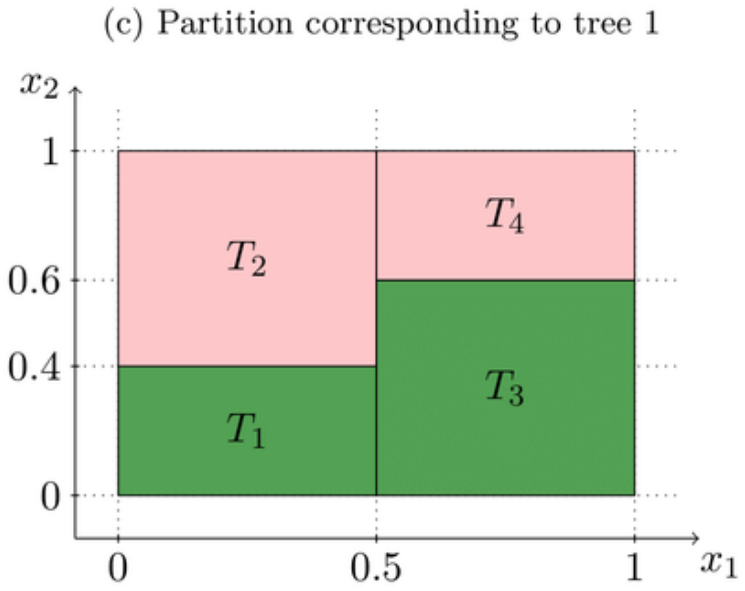
- Partition Trees
  - Hierarchical space partition

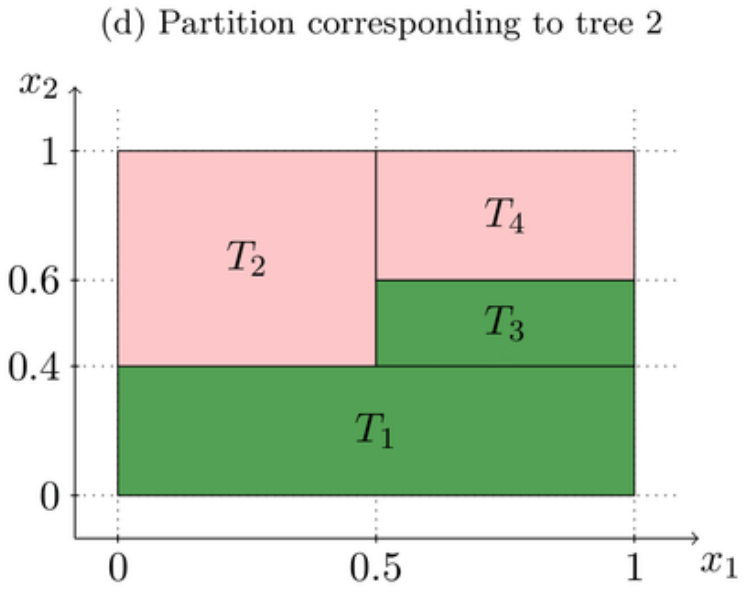
    Here, line partitions space into left and right subtree

  - Each node is associated with a region









Source: A toolkit for stability assessment of tree-based learners

Point Quadtree

- Suppose that we wish to store a set $P = \{p_1, \ldots, p_n\}$ of n points in d-dimensional space.

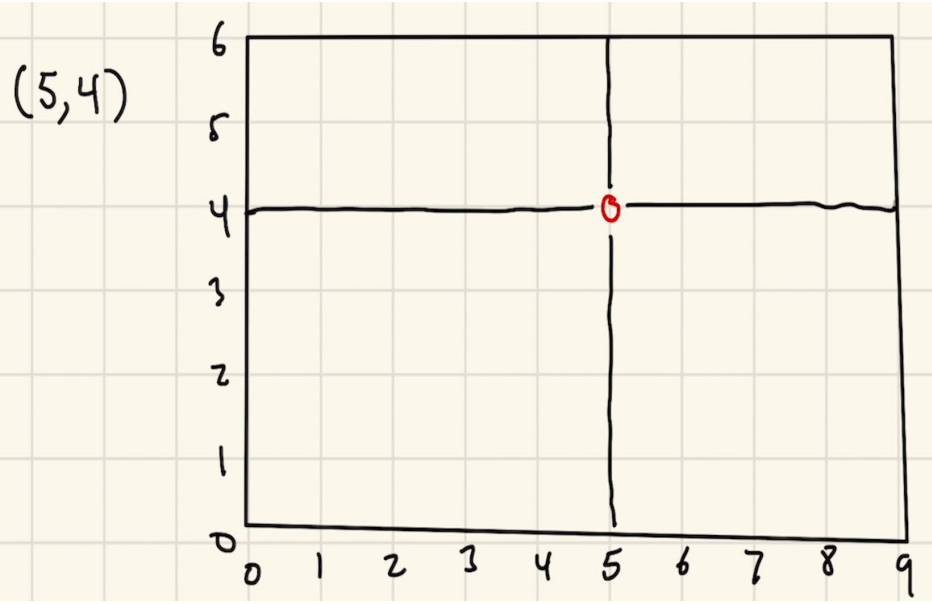
- In binary trees, each point naturally splits the real line in two. In two dimensions if we run a vertical and horizontal line through the point, it naturally subdivides the plane into four quadrants about this point.

- We look at the 2-dimension space : point quadtree

Point Quadtree

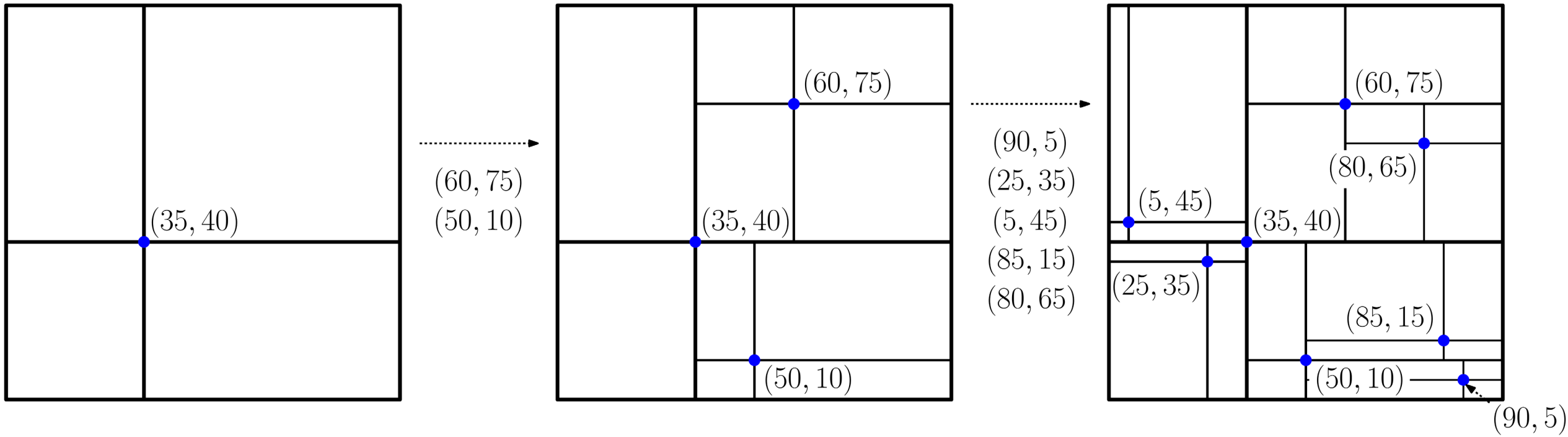
- Each node has four (possibly null) children, corresponding to the four quadrants defined by the 4-way subdivision. We label these according to the compass directions, as NW, NE, SW, and SE.

- We descend through the tree structure in a natural way. For example, we compare the newly inserted point's x and y coordinates to those of the root. If the x is larger and the y is smaller, we recurse on the SE child. The insertion of each point results in a subdivision of a rectangular region into four smaller rectangles.

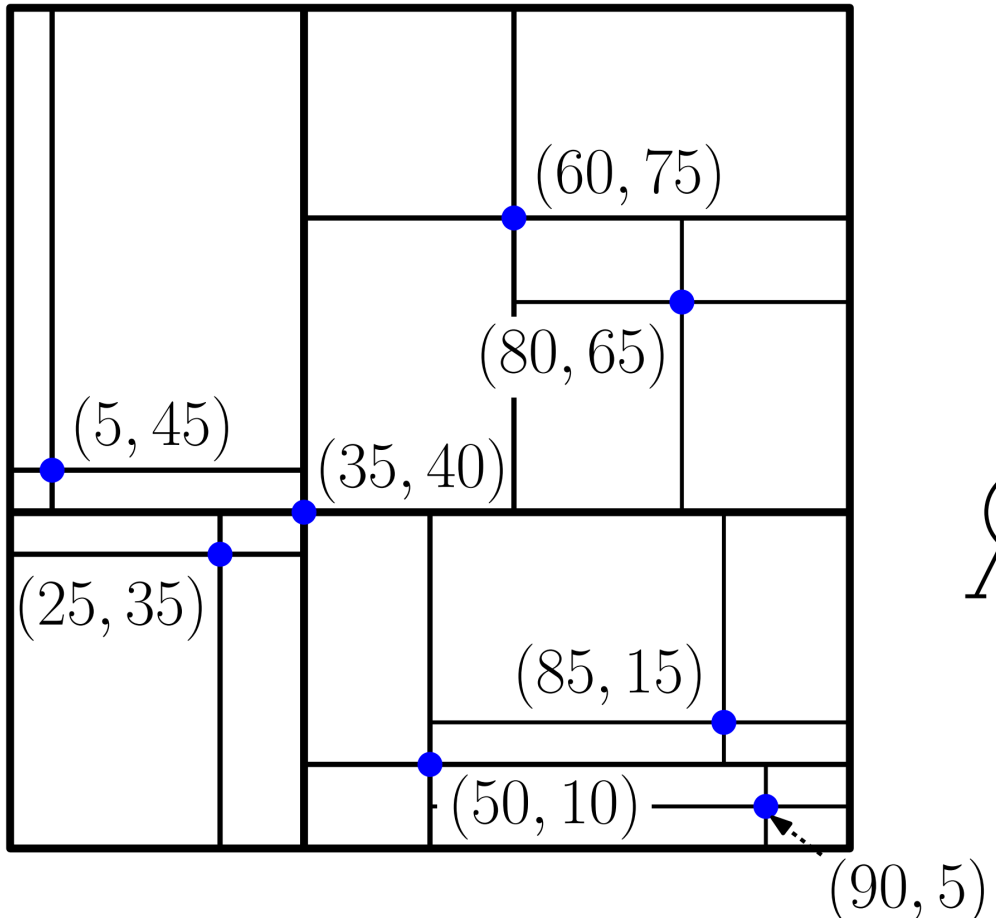


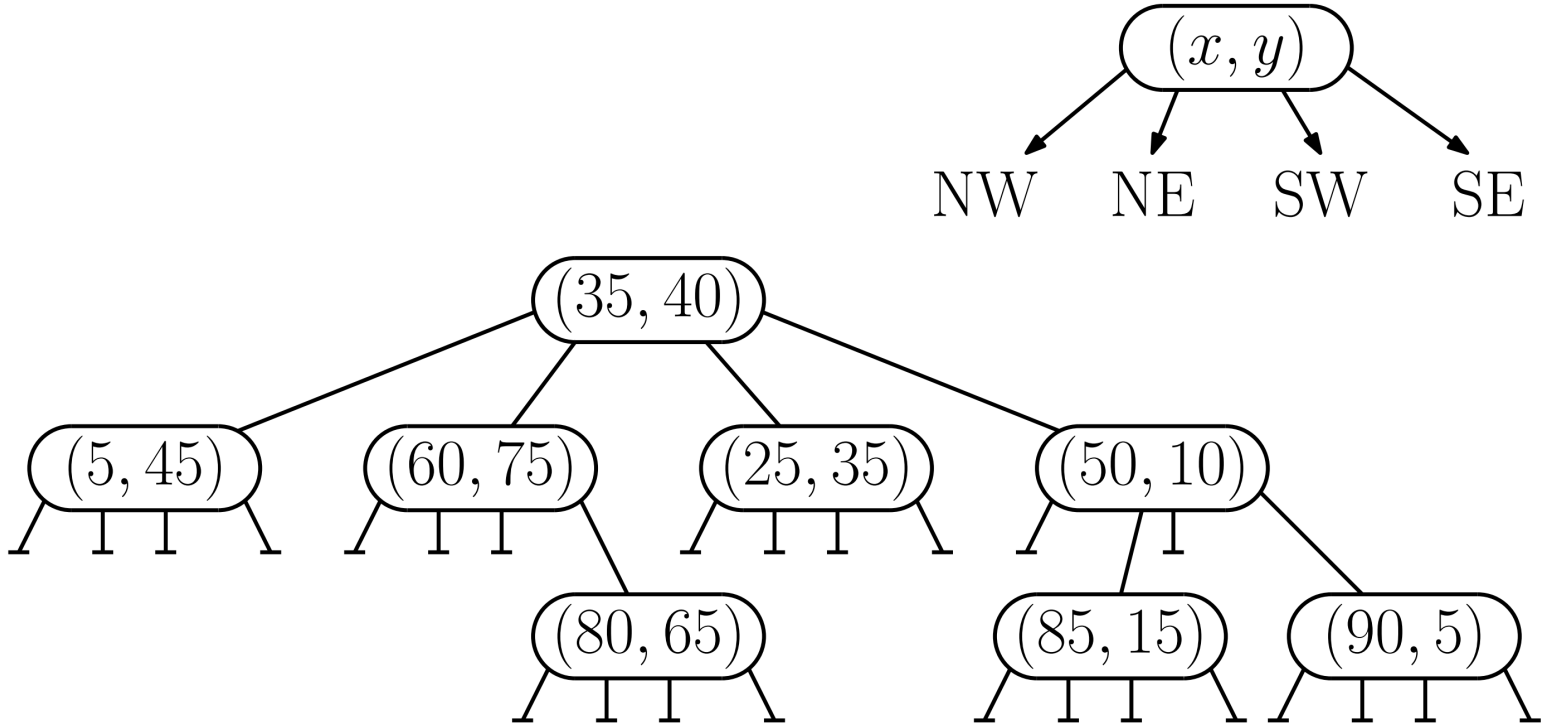
Point Quadtree Subdivision

- Insertion of the points: (35, 40), (50, 10), (60, 75), (80, 65), (85, 15), (5, 45), (25, 35), (90, 5)



Subdivision and corresponding Tree





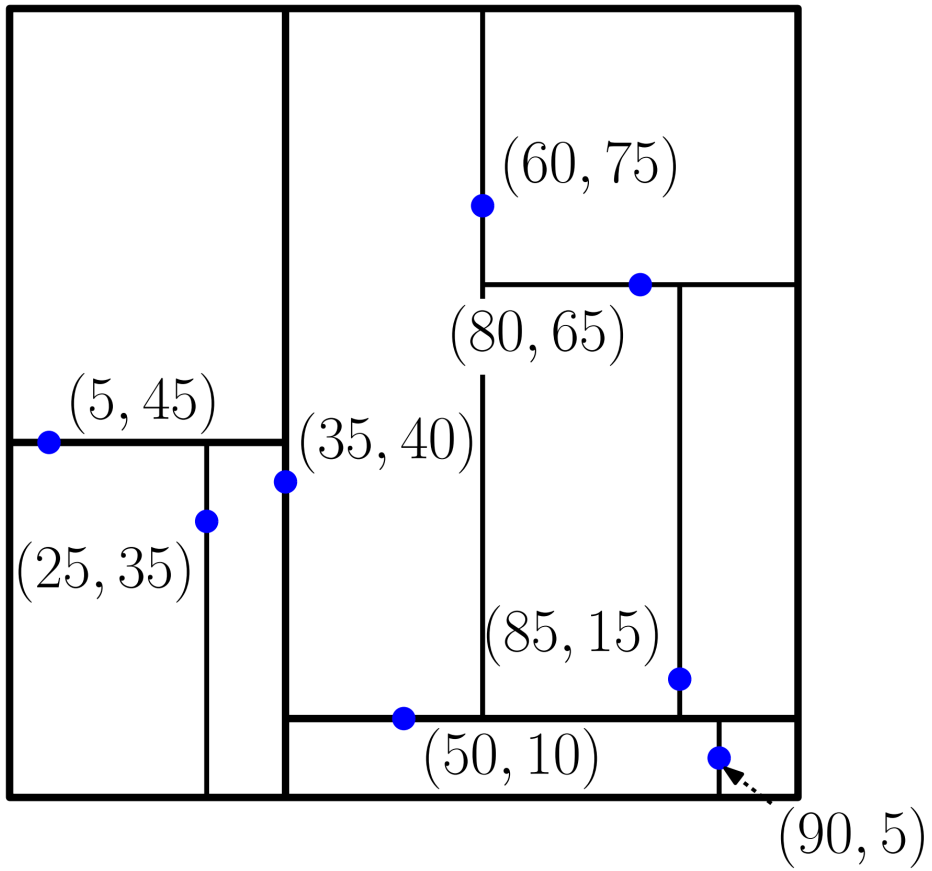
Point kd-tree

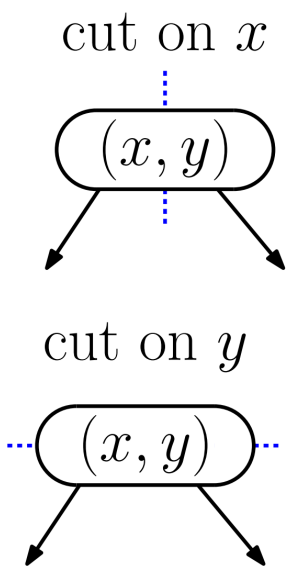
- Point quadtrees can be generalized to higher dimensions, the number of children grows exponentially in the dimension, as 2d.
  - For example, if we are working in 20-dimensional space, every node has 2^{20}, or roughly a million children! Clearly, the simple quadtree idea is not scalable to very high dimensions.

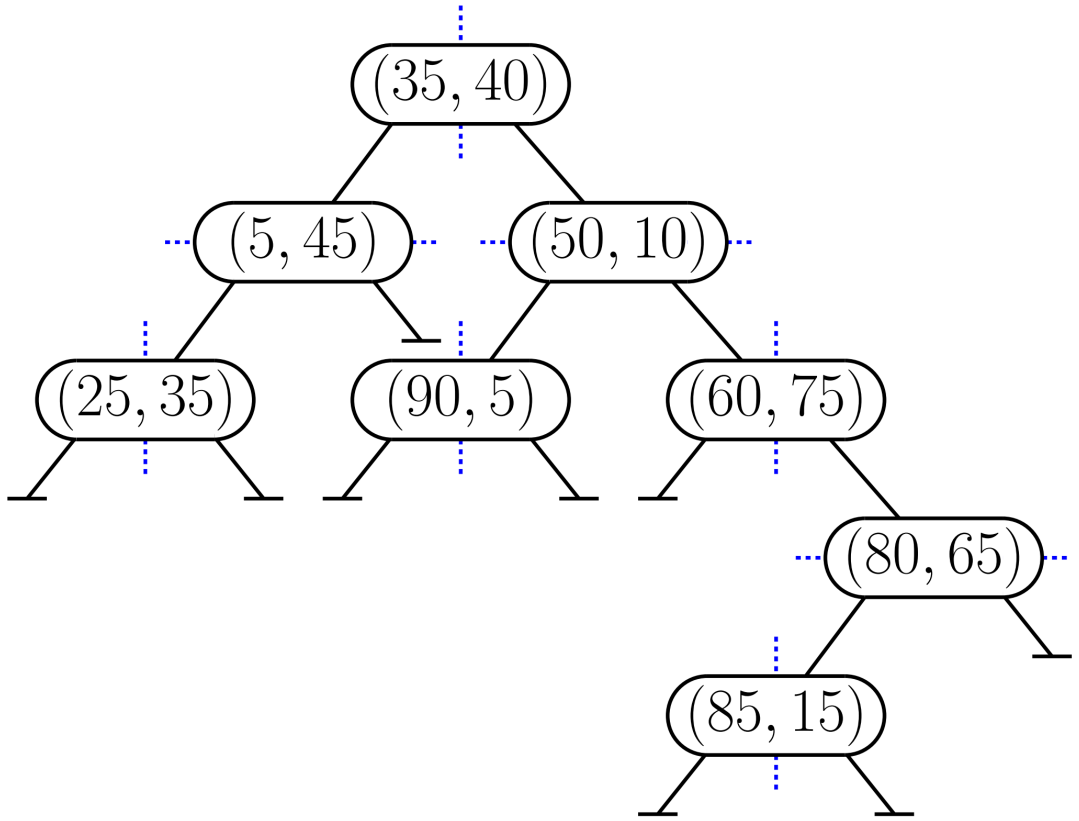
- Like in a quadtree, when a new point is inserted into some leaf node's cell, we split the cell by a horizontal or vertical splitting line, which passes through this point
    - Often we alternate between the horizontal and the vertical

Point kd-tree

- Height of the tree: O(lg n)







Data Structure

```
class KDNode {                                  // node in a kd-tree
    Point point                                 // splitting point
    int cutDim                                  // cutting dimension (optional)
    KDNode left, right                          // children

    KDNode(Point point, int cutDim) {           // constructor
        this.point = point
        this.cutDim = cutDim
        left = right = null
    }

    boolean inLeftSubtree(Point pt) {           // is pt in left subtree?
        return pt[cutDim] < point[cutDim]
    }
}
```

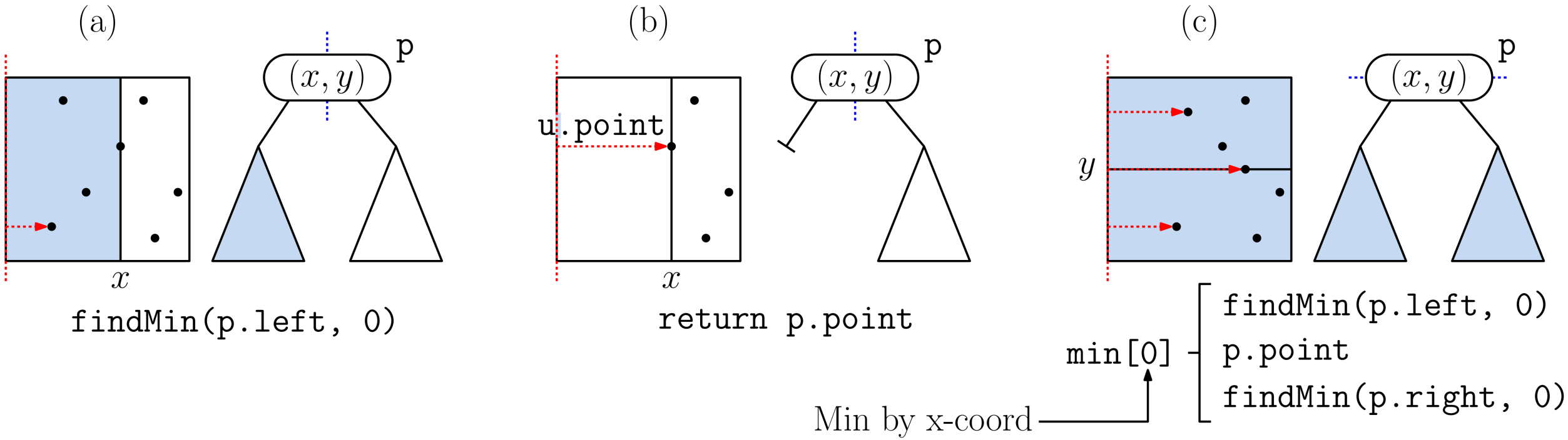
Insertion

- The function takes three arguments, the point **pt** being inserted, the current node **p**, and the cutting dimension **cd** of the newly created node. The initial call is root = insert(pt, root, 0).

```
KDNode insert(Point pt, KDNode p, int cd) { // insert pt in subtree p with cutDim cd
    if (p == null) {                                    // fell out of tree
        p = new KDNode(pt, cd)                          // create new leaf
    } else if (p.point.equals(pt)) {
        throw Exception("Error - duplicate point")
    } else if (p.inLeftSubtree(pt)) {                   // insert into left subtree
        p.left = insert(pt, p.left, (cd + 1) % dim)
    } else {                                            // insert into right subtree
        p.right = insert(pt, p.right, (cd + 1) % dim)
    }
    return p
}
```

Deletion

- How to find the replacement for the deleted node?



Deletion

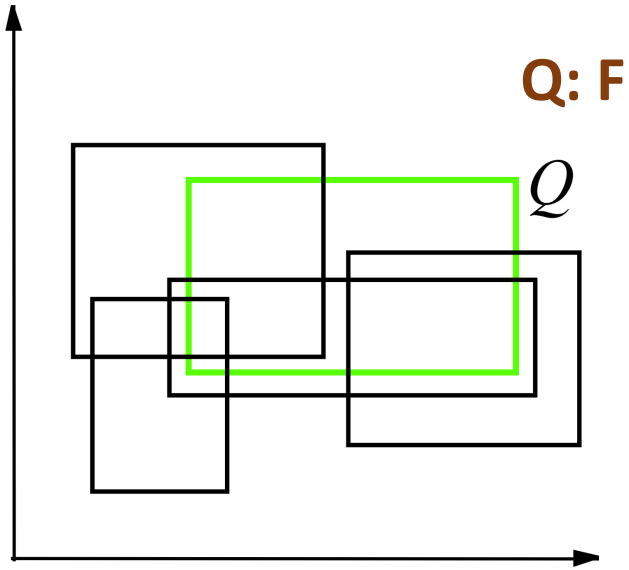
- Finding replacement for the deletion

```
Point findMin(KDNode p, int i) {                    // get min point along dim i
    if (p == null) return null                      // fell out of tree?
    if (p.cutDim == i) {                            // cutting dimension matches i?
        if (p.left == null)                         // no left child?
            return p.point                          // use this point
        else
            return findMin(p.left, i)               // get min from left subtree
    } else {                                        // it may be in either side
        return min[i](findMin(p.left), p.point, findMin(p.right))
    }
}
```

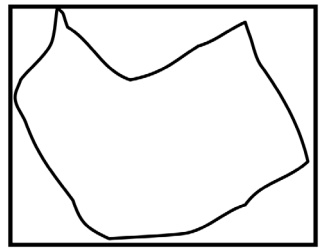
```
KDNode delete(Point pt, KDNode p) {
    if (p == null) {                                  // fell out of tree?
        throw Exception("Error - Point does not exist");
    } else if (p.point.equals(pt)) {                  // found it
        if (p.right != null) {                        // can replace from right
            p.point = findMin(p.right, p.cutDim)      // find and copy replacement
            p.right = delete(p.point, p.right)        // delete from right
        } else if (p.left != null) {                  // can replace from left
            p.point = findMin(p.left, p.cutDim)       // find and copy replacement
            p.right = delete(p.point, p.left)         // delete left but move to right!!
            p.left = null                             // left subtree is now empty
        } else {                                      // deleted point in leaf
            p = null                                  // remove this leaf
        }
    } else if (p.inLeftSubtree(pt)) {
        p.left = delete(pt, p.left)                   // delete from left subtree
    } else {                                          // delete from right subtree
        p.right = delete(pt, p.right)
    }
    return p
}
```

Rectangle Range Searching

- Report all rectangles intersecting query rectangle Q



Q: Find all restaurants in Sanepa

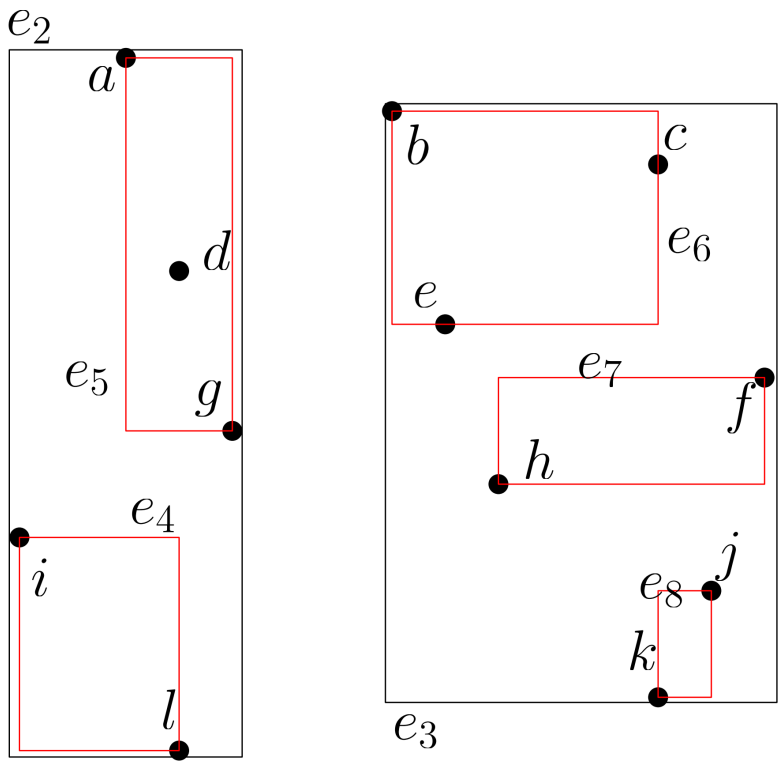


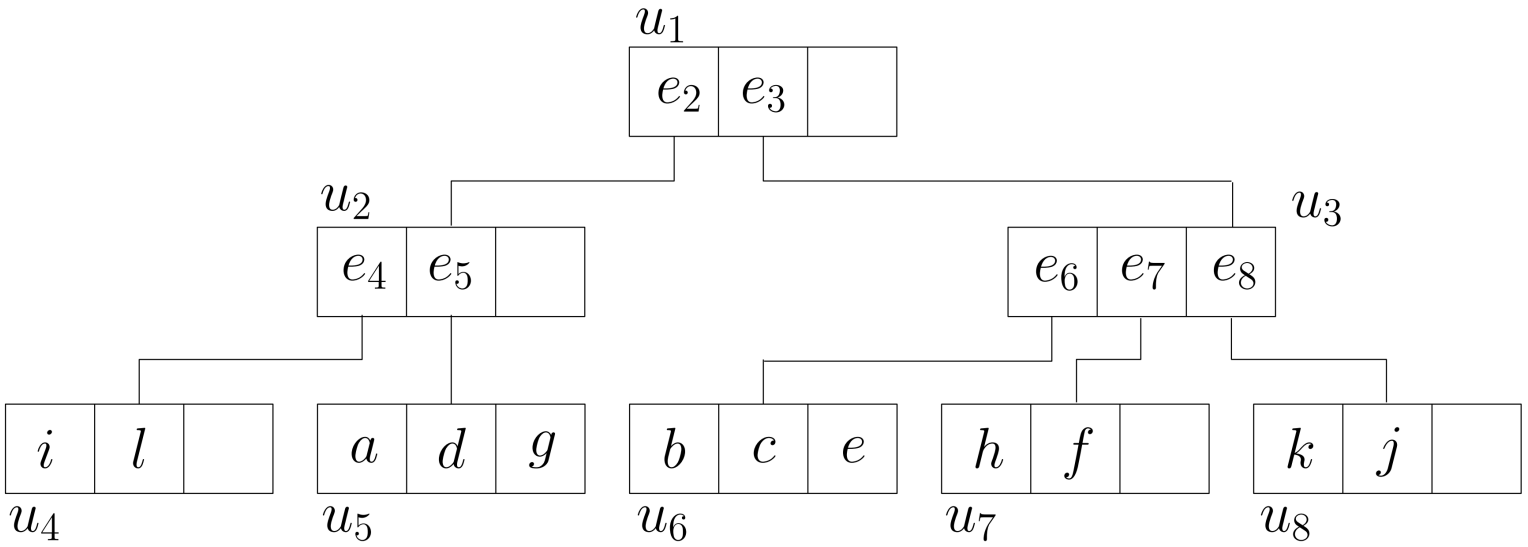
- Often used in practice when handling complex geometric objects
  - Store minimal bounding rectangles (MBR)

R-trees

Similar to B-tree

- Rectangles in leaves (on same level)
- Internal nodes contain MBR of rectangles below each child





R-trees

- The key idea of the data structure is to group nearby objects and represent them with their minimum bounding rectangle in the next higher level of the tree

- Since all objects lie within this bounding rectangle, a query that does not intersect the bounding rectangle also cannot intersect any of the contained objects

- At the leaf level, each rectangle describes a single object

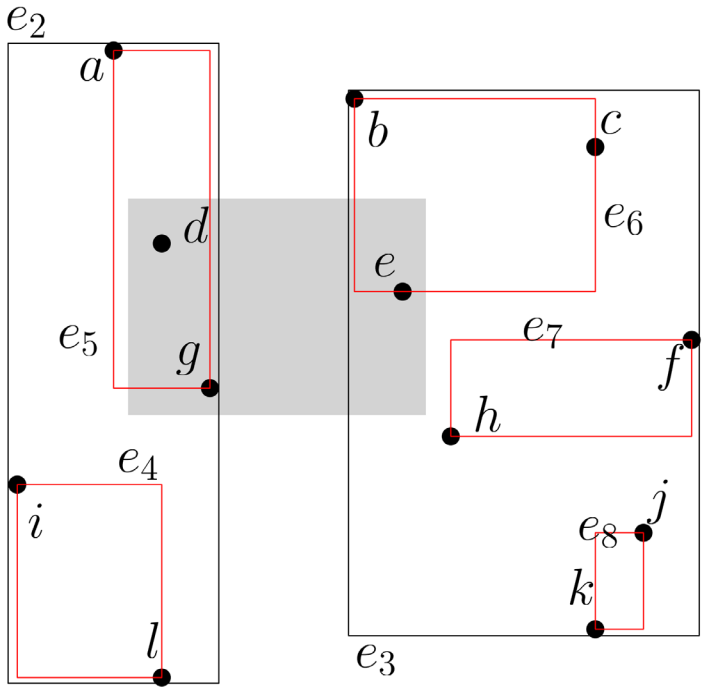
Search

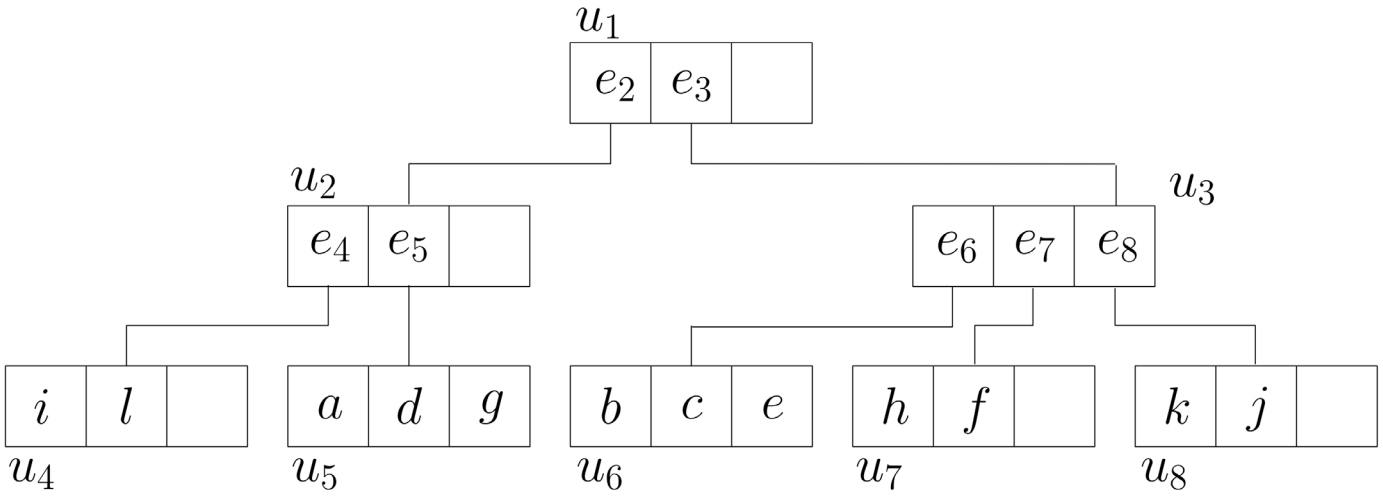
Let q be the search region of a range query. We invoke **range-query**(root, q), where root is the root of the tree

range-query(u, r)
if u is a leaf then
    report all points stored at u that are covered by r
else
    for each child v of u do
        if MBR(v) intersects r then
            range-query(v, r)

Example

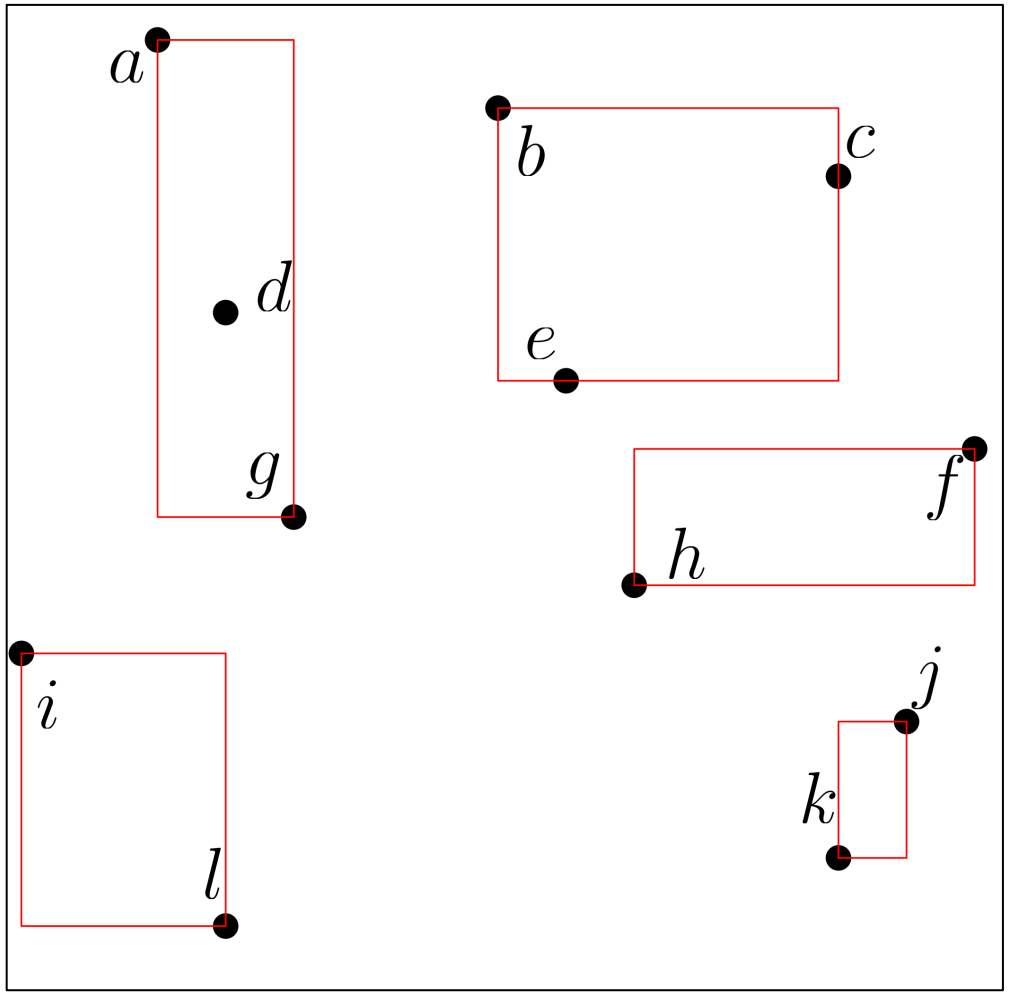
- Nodes u1, u2, u3, u5, u6 are accessed to answer the query with the shaded search region

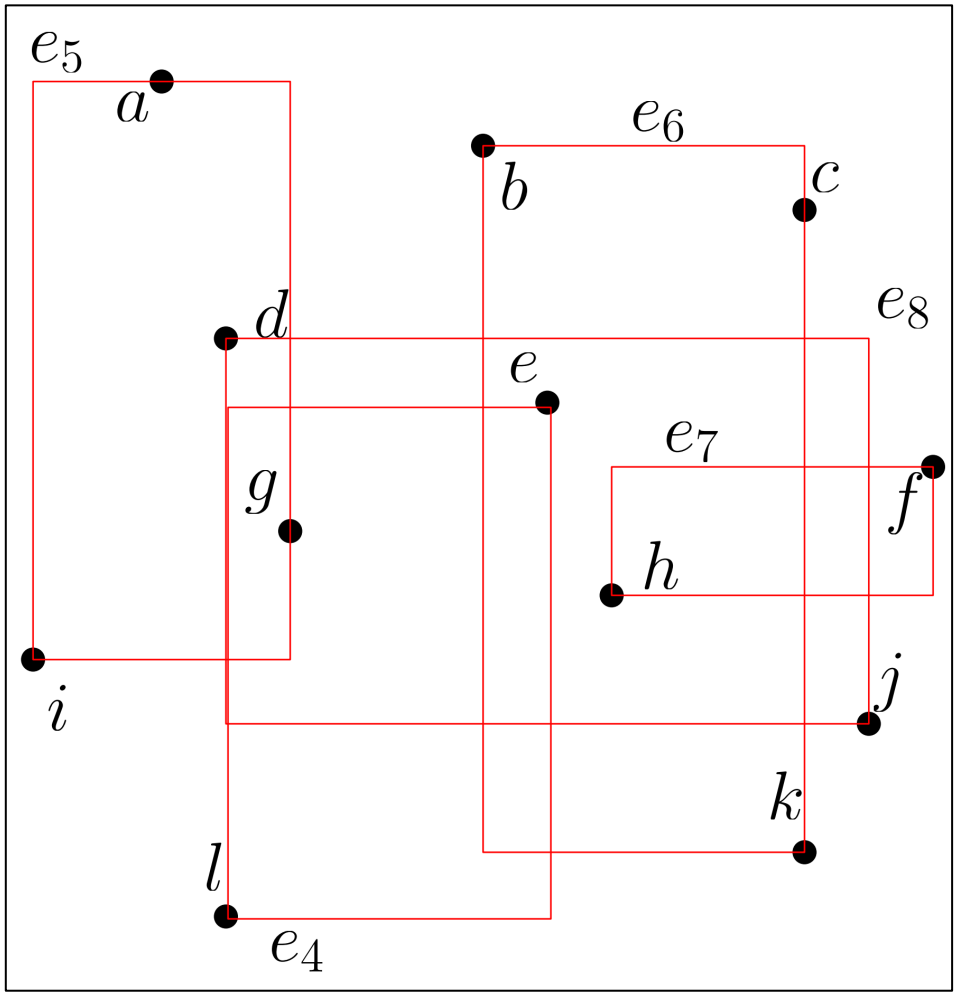




Tree Construction

Which is better?

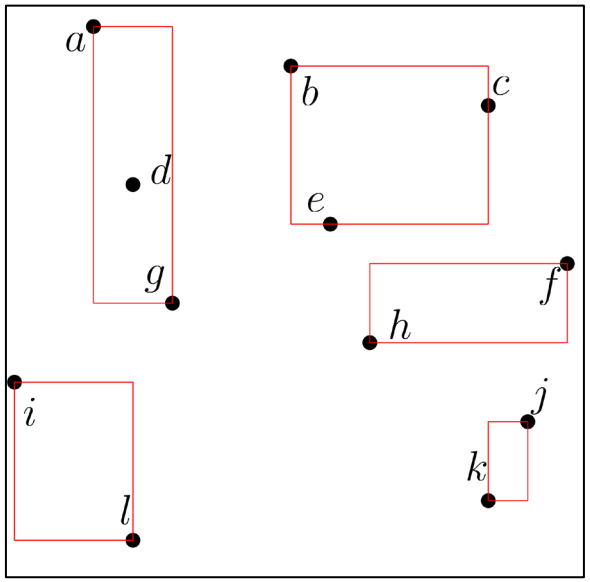


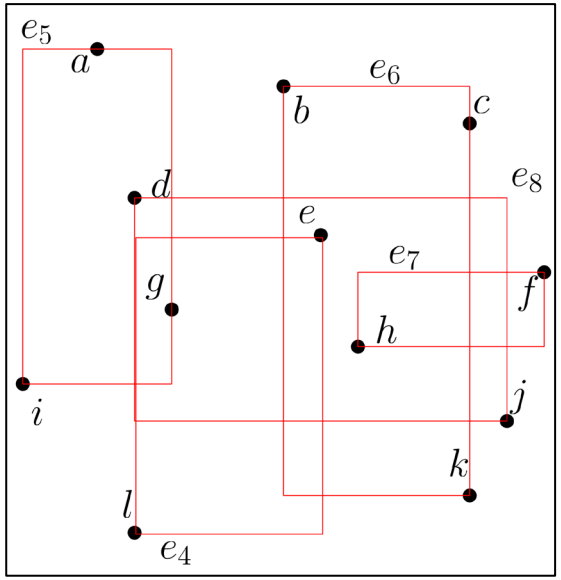


Common Principle

- In general, the construction algorithm of the R-tree aims at minimizing the perimeter sum of all the MBRs.

- For example, the left tree has a smaller perimeter sum than the right one.





Insertion

- Let p be the point being inserted and root is the root of the tree

```
insert(u, p)
if u is a leaf node then
        add p to u
        if u overflows then
        /* namely, u has B + 1 points */
        handle-overflow(u)
else
        v choose-subtree(u, p)
        /* which subtree under u should we insert p into? */
        insert(v, p)
```

Overflow

handle-overflow(*u*)
    split(*u*) into *u* and *u'*
    if *u* is the root **then**
        create a new root with *u* and *u'* as its child nodes
    else
        w ← the parent of *u*
        update *MBR*(*u*) in *w*
        add *u'* as a child of *w*
        if *w* overflows **then**
            handle-overflow(*w*)

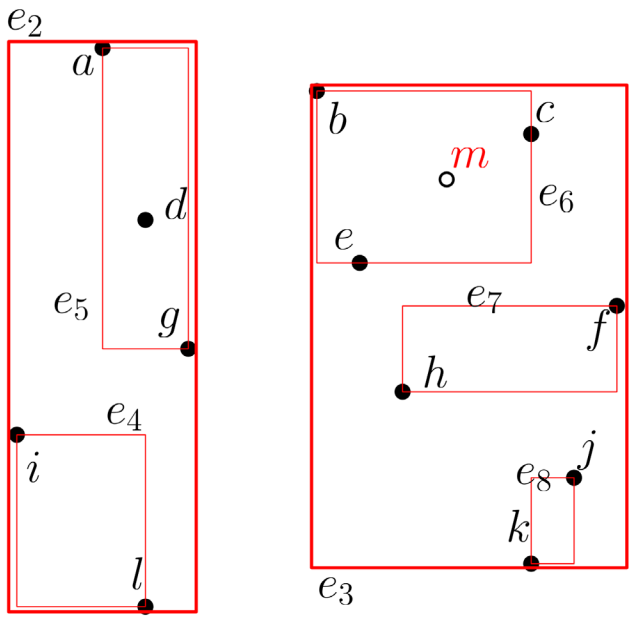
Choosing a subtree

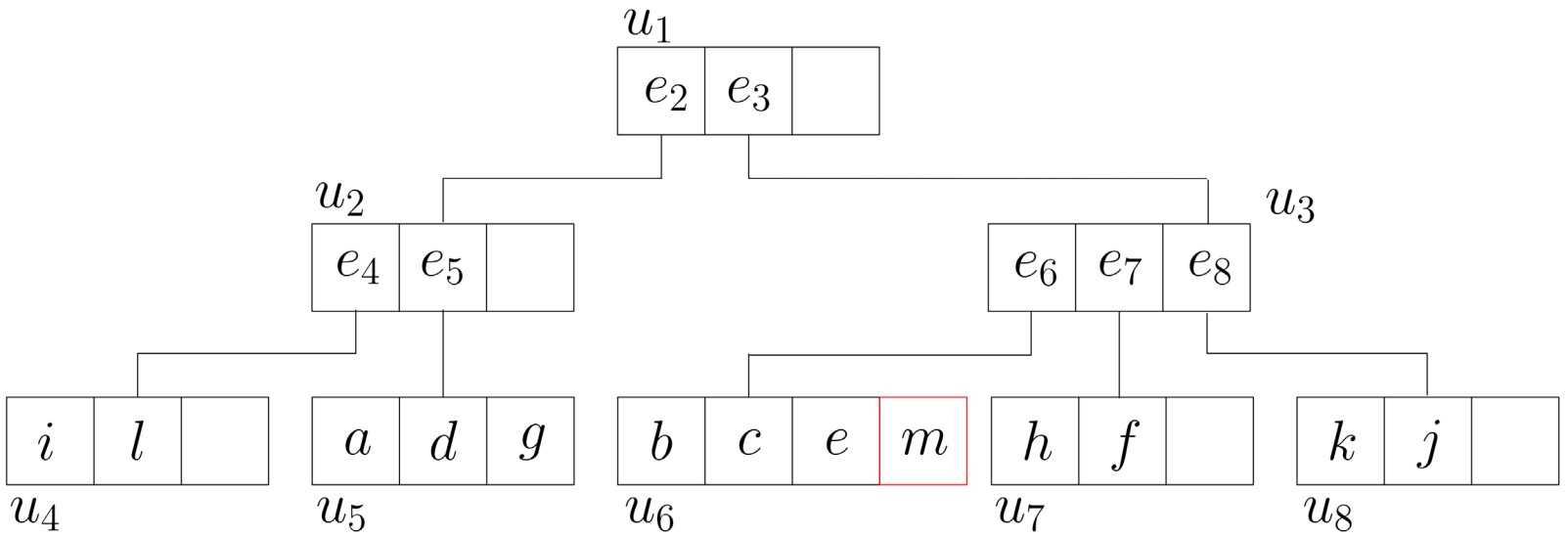
choose-subtree(*u, p*)

return the child whose MBR requires the minimum increase in perimeter to cover *p*.
break ties by favoring the smallest MBR.

Example

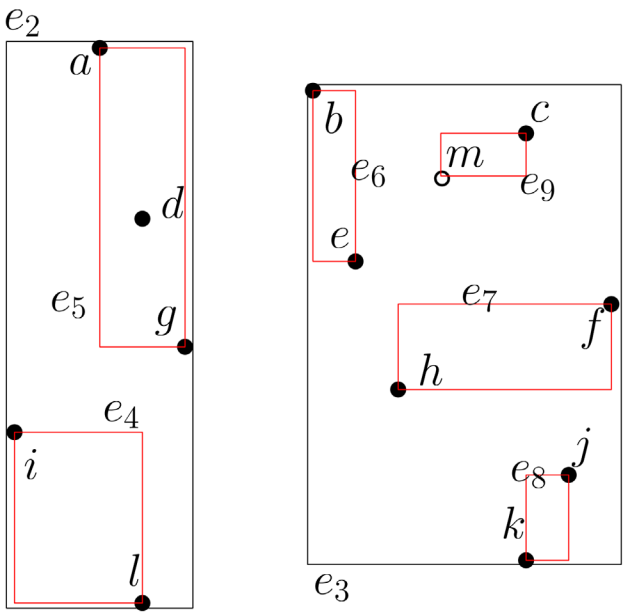
Assume that we want to insert the white point **m**. By applying choose-subtree twice, we reach the leaf node **u6** that should accommodate m. The node overflows after incorporating m (here, B = 3)

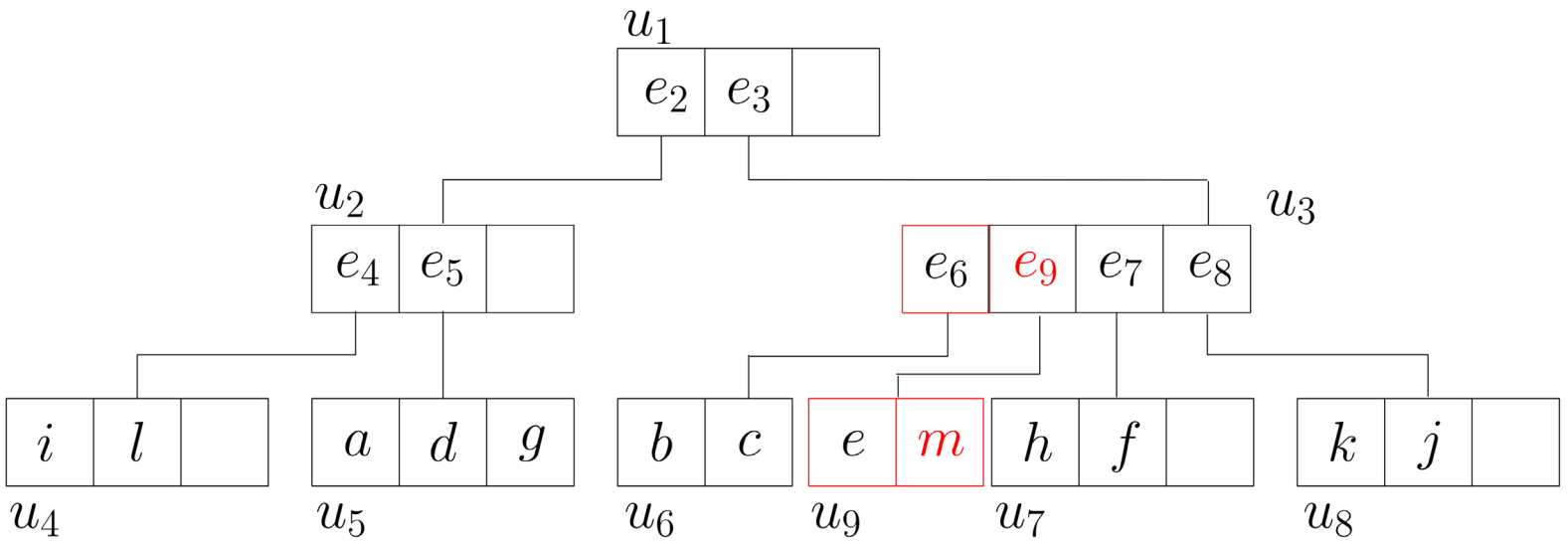




Example (contd.)

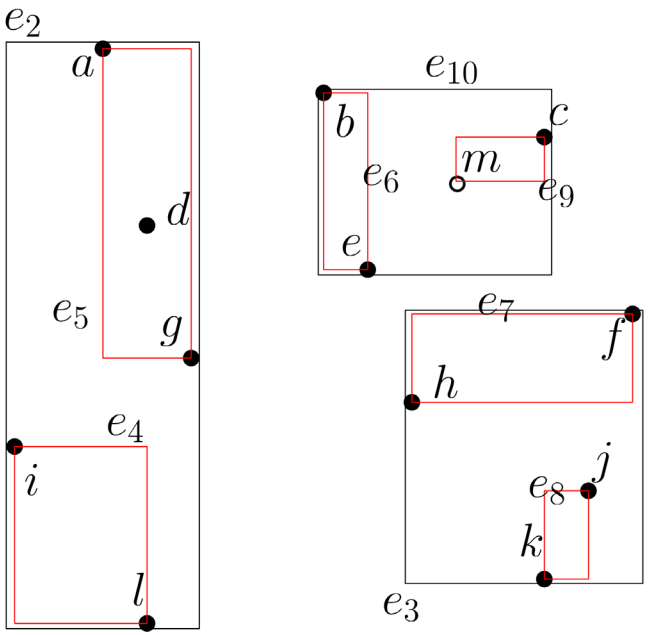
Node u_6 splits, generating u_9. Adding u_9 as a child of u_3 causes u_3 to overflow

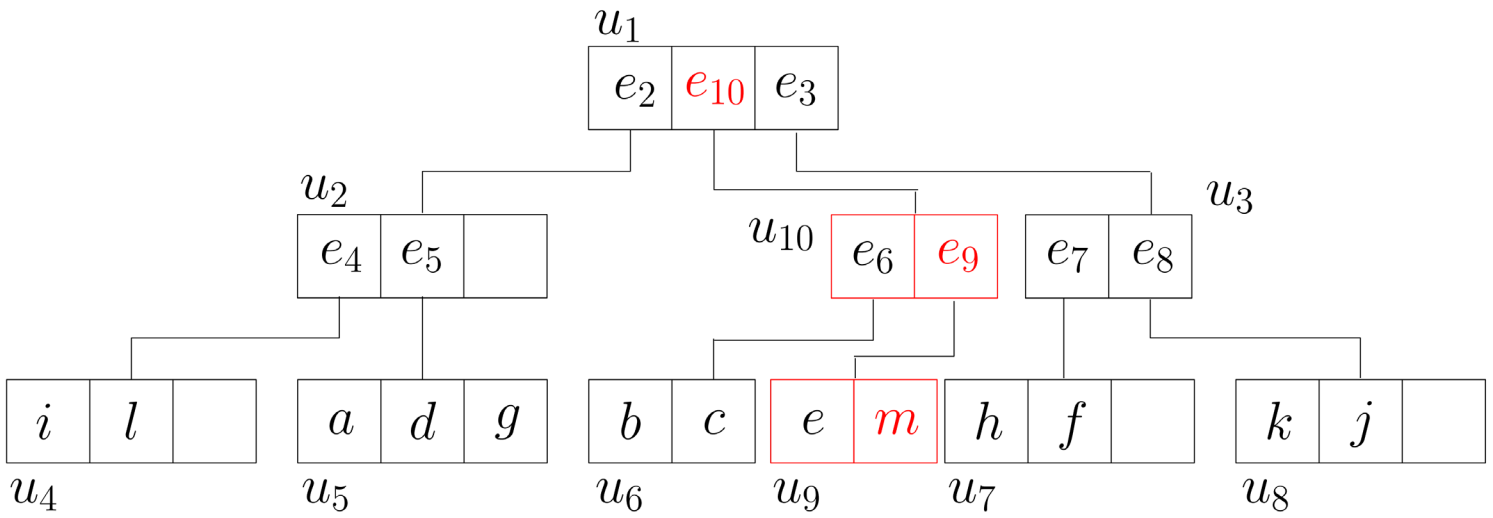




Example (contd.)

Node u_3 splits, generating u_{10}. The insertion finishes after adding u_{10} as a child of the root

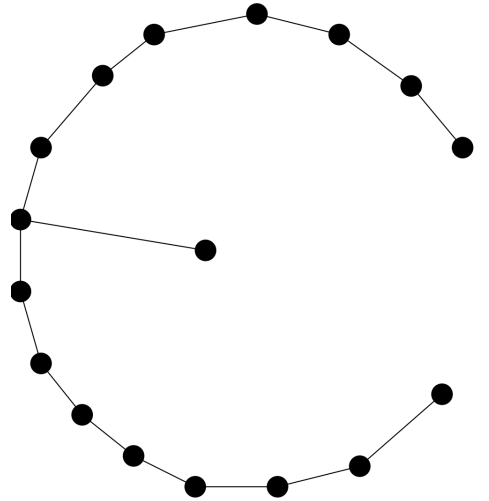


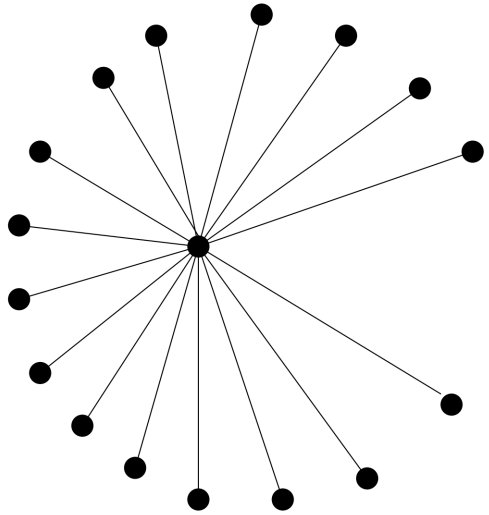


Minimum Spanning Trees

- A tree is a connected graph with no cycles. A spanning tree is a subgraph of G which has the same set of vertices of G and is a tree.

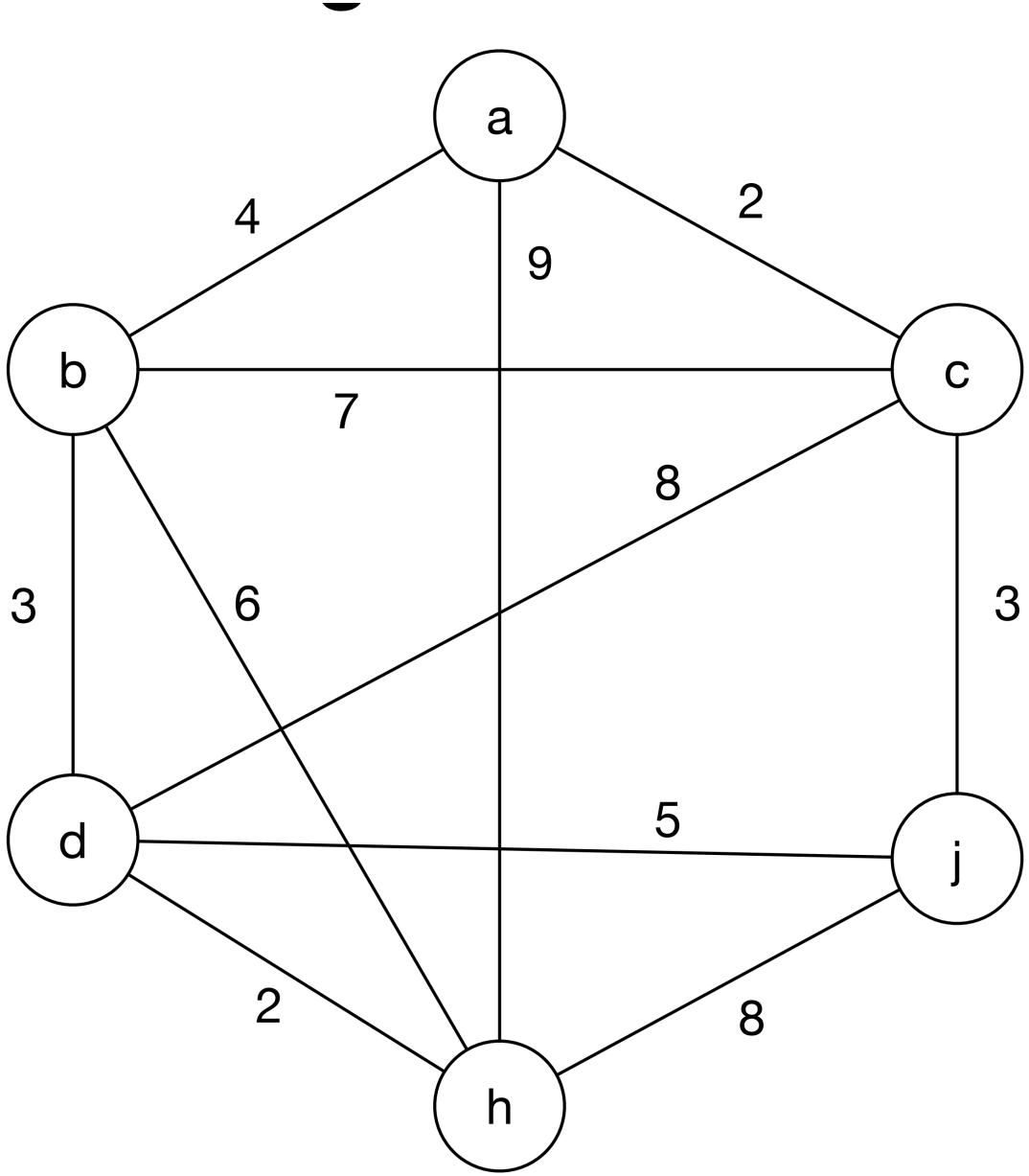
- A minimum spanning tree of a weighted graph G is the spanning tree of G whose edges sum to minimum weight. There can be more than one minimum spanning tree in a graph - consider a graph with identical weight edges.





Prim’s Algorithm

```
function prims(G)
        let E' = {}
        pick any starting node, s, in V
        let V' = {s}
        while |V'| < |V|
                find, e, the minimum weight edge with one endpoint in V’
                add this edge to E’
                add the endpoint of e not in V' to V’
        return (V', E')
```

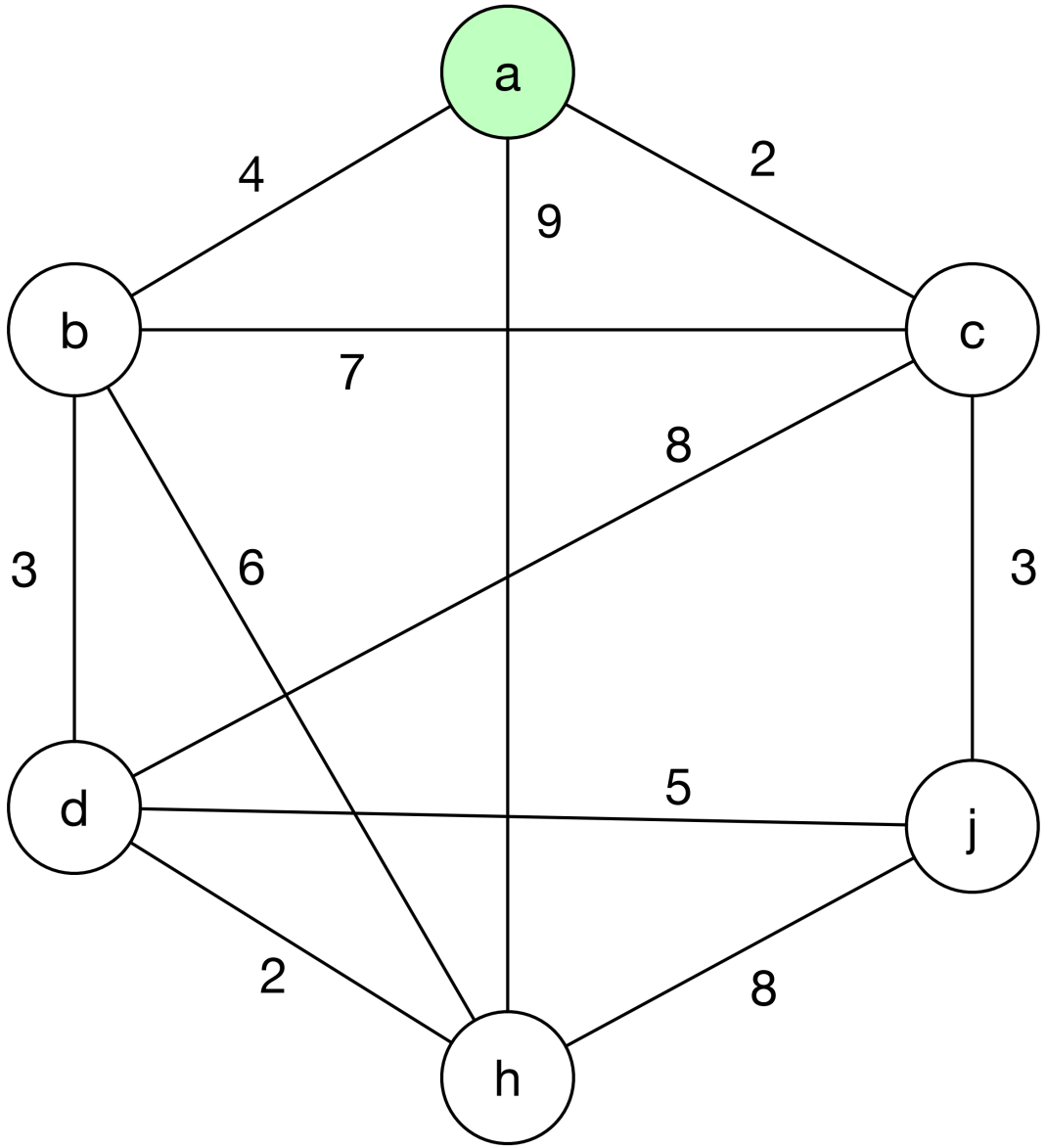


V = {a, b, c, d, j, h}

E = {ab, ac, ah, bc, bd,
bh, cj, cd, dj, dh, jh}

V' = {}

E' = {}

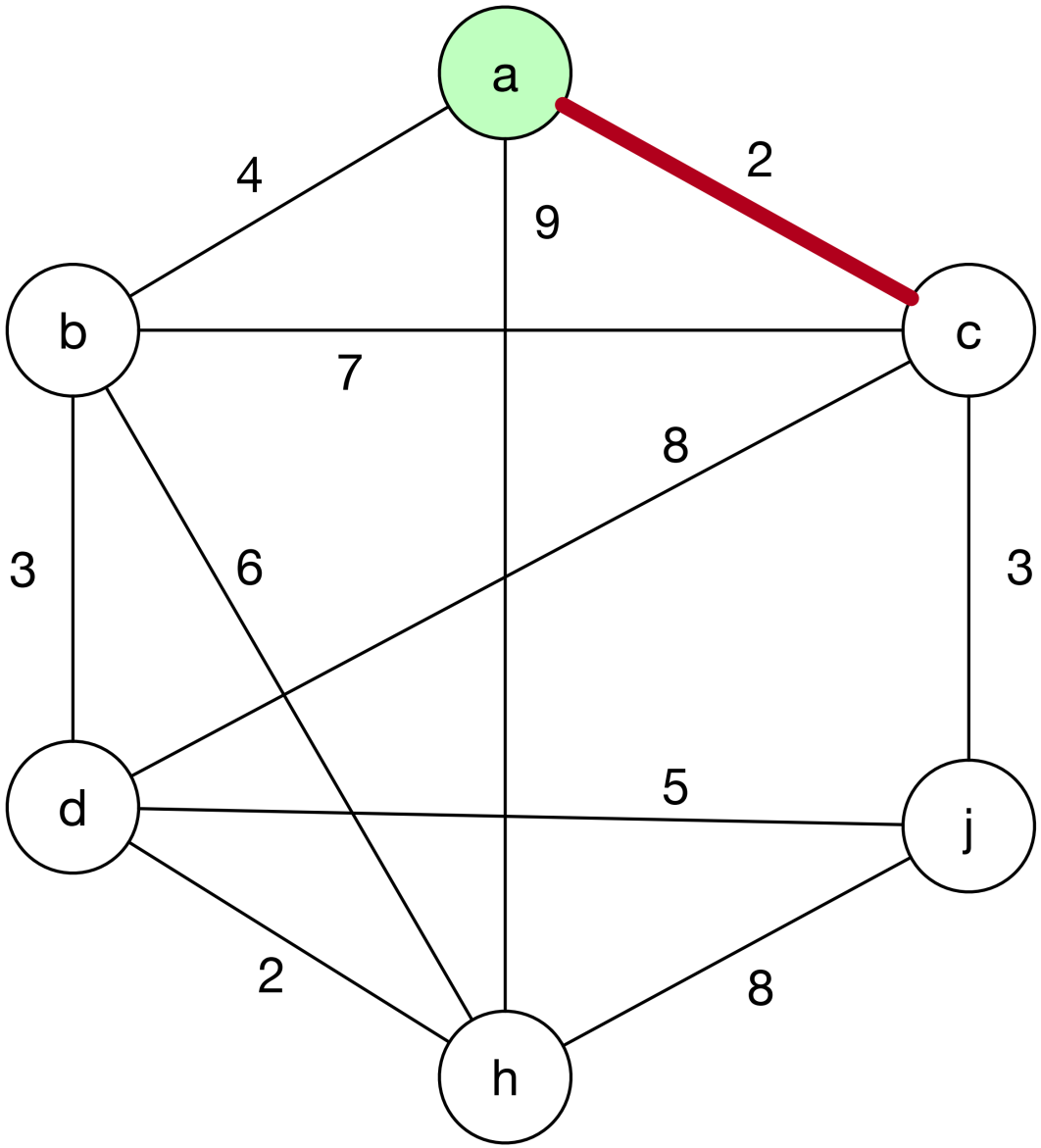


V = {a, b, c, d, j, h}

E = {ab, ac, ah, bc, bd,
bh, cj, cd, dj, dh, jh}

V' = {a}

E' = {}

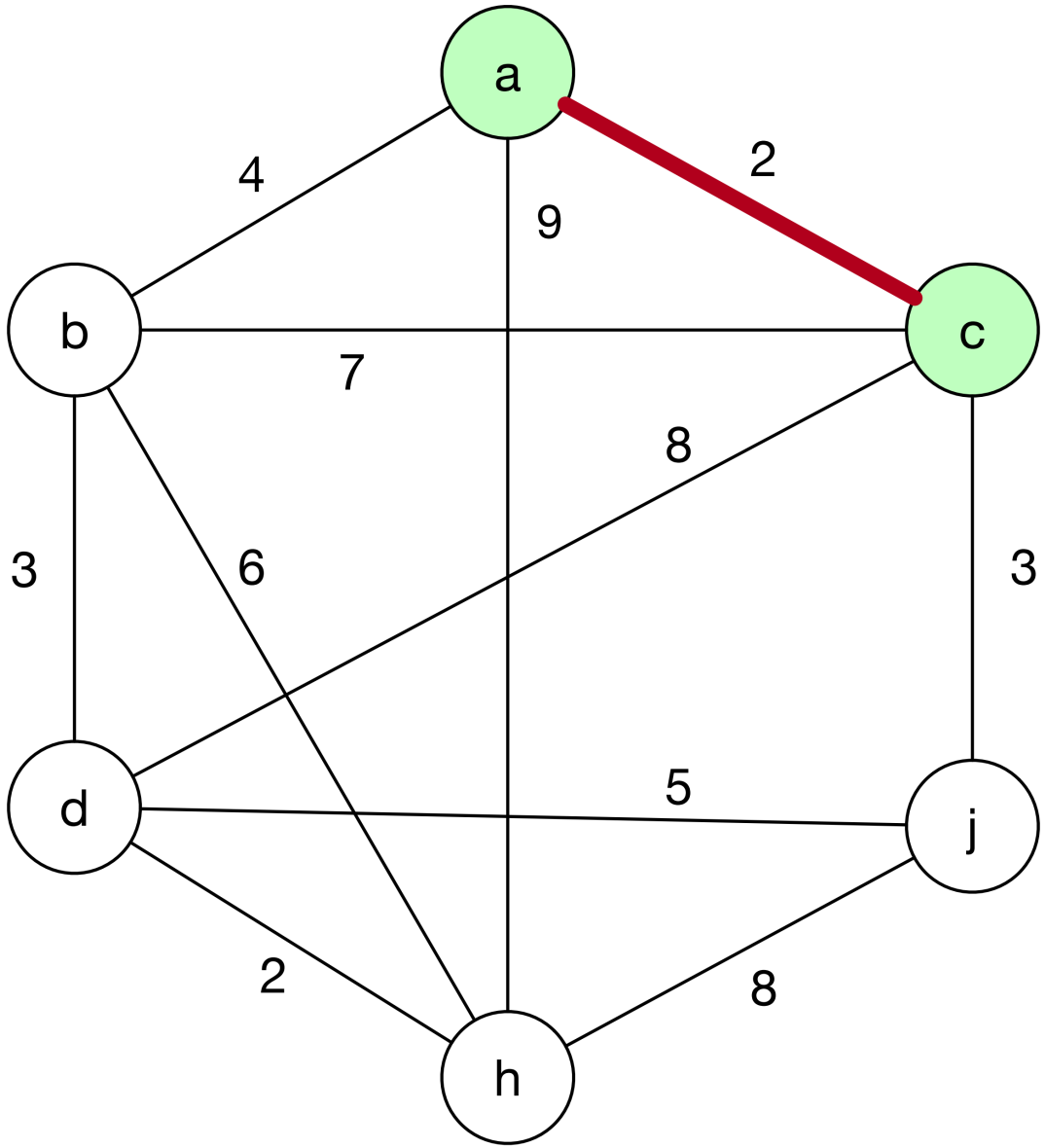


V = {a, b, c, d, j, h}

E = {ab, ac, ah, bc, bd,
bh, cj, cd, dj, dh, jh}

V' = {a}

E' = {ac}

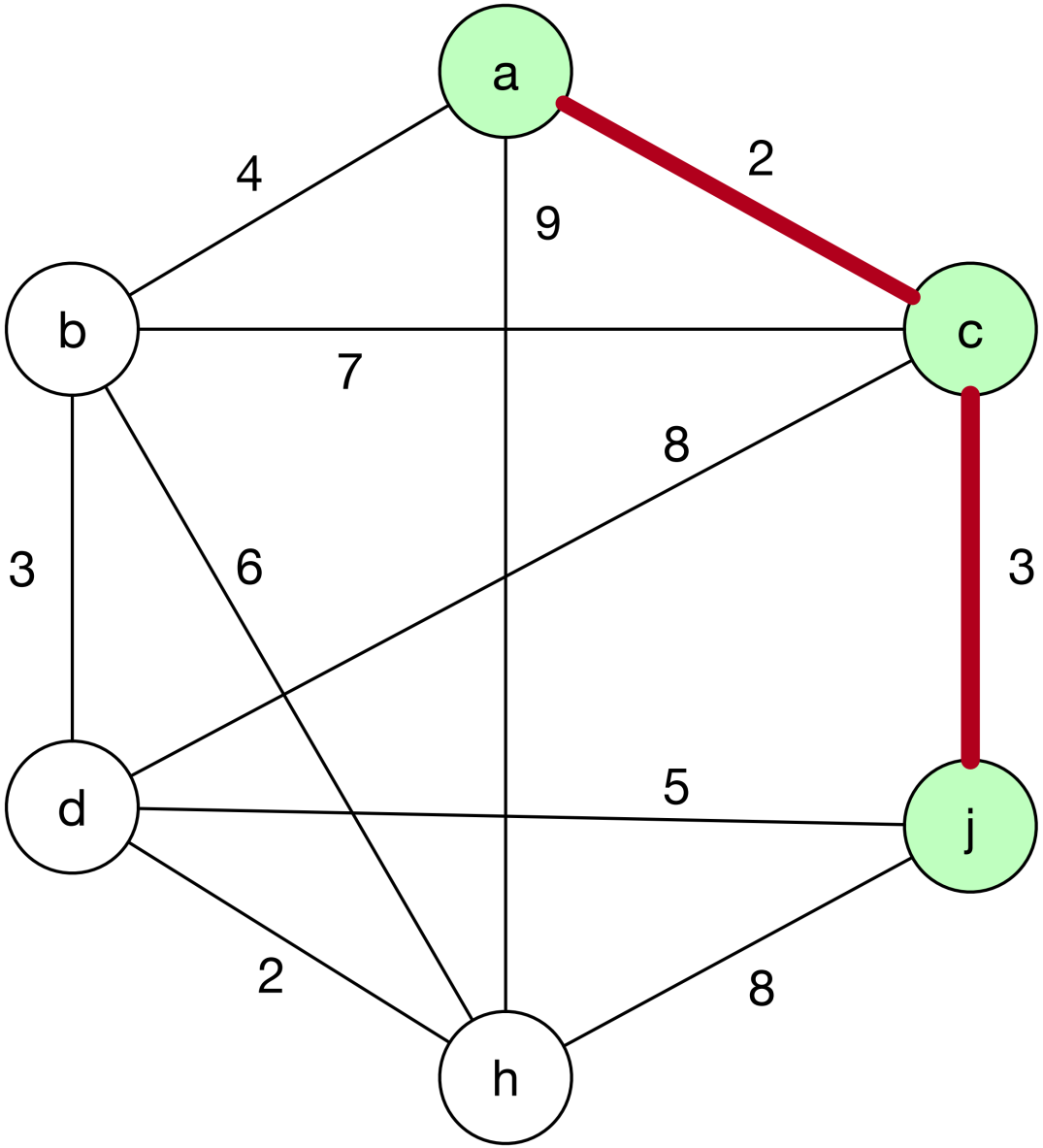


V = {a, b, c, d, j, h}

E = {ab, ac, ah, bc, bd,
bh, cj, cd, dj, dh, jh}

V' = {a, c}

E' = {ac}

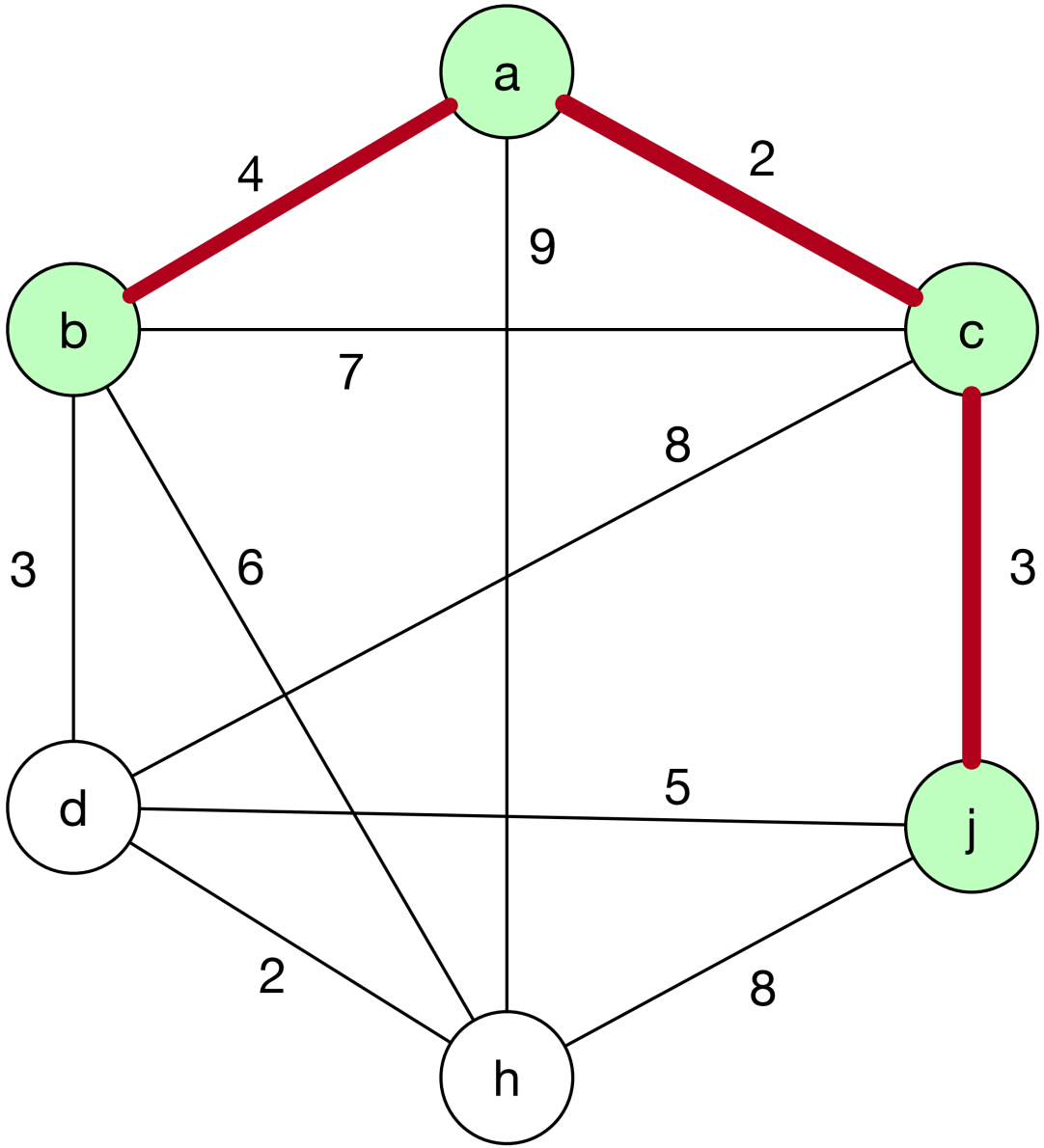


V = {a, b, c, d, j, h}

E = {ab, ac, ah, bc, bd,
bh, cj, cd, dj, dh, jh}

V' = {a, c, j}

E' = {ac, cj}

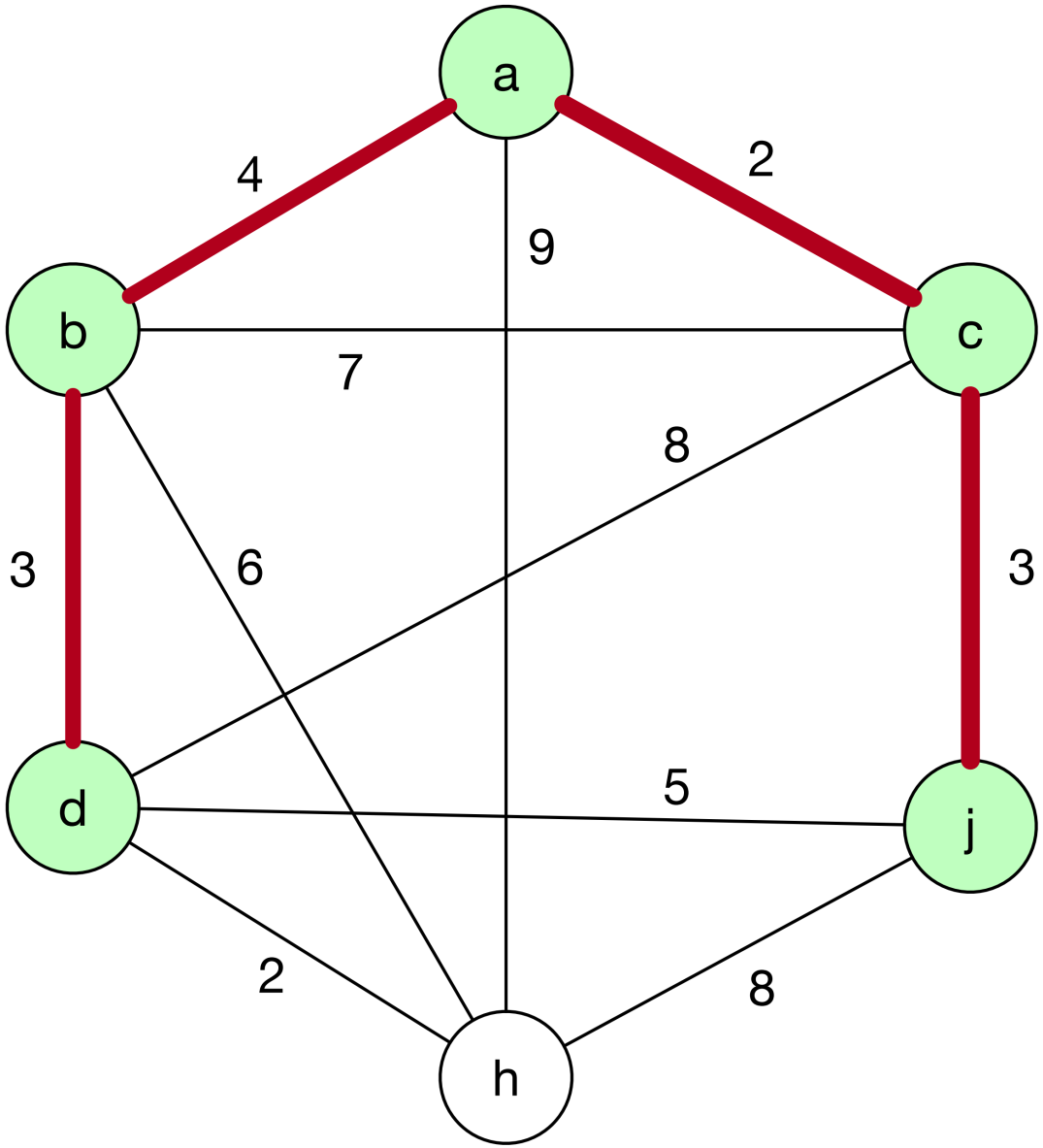


V = {a, b, c, d, j, h}

E = {ab, ac, ah, bc, bd,
bh, cj, cd, dj, dh, jh}

V' = {a, c, j, b}

E' = {ac, cj, ab}

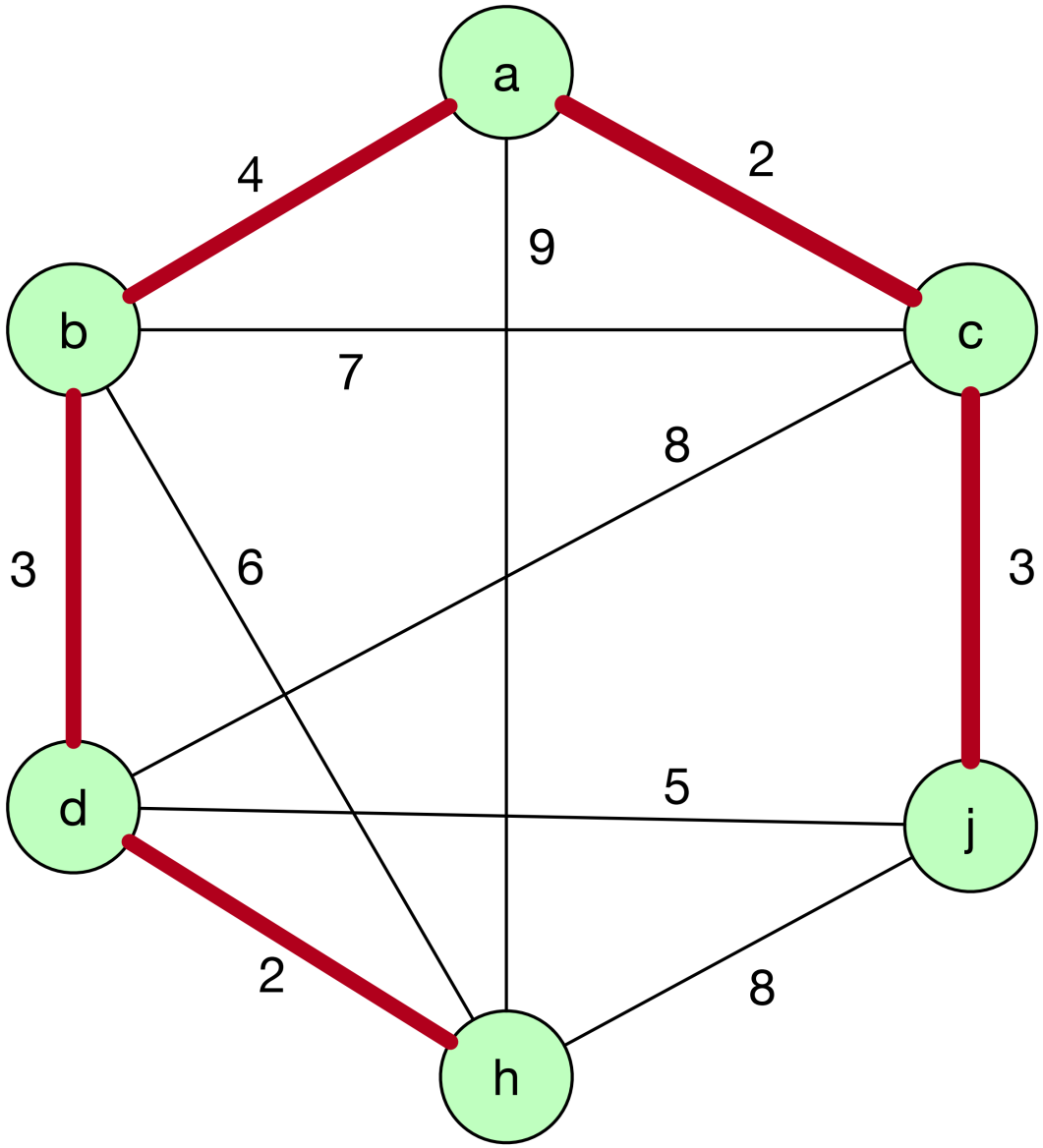


V = {a, b, c, d, j, h}

E = {ab, ac, ah, bc, bd,
bh, cj, cd, dj, dh, jh}

V' = {a, c, j, b, d}

E' = {ac, cj, ab, bd}



V = {a, b, c, d, j, h}

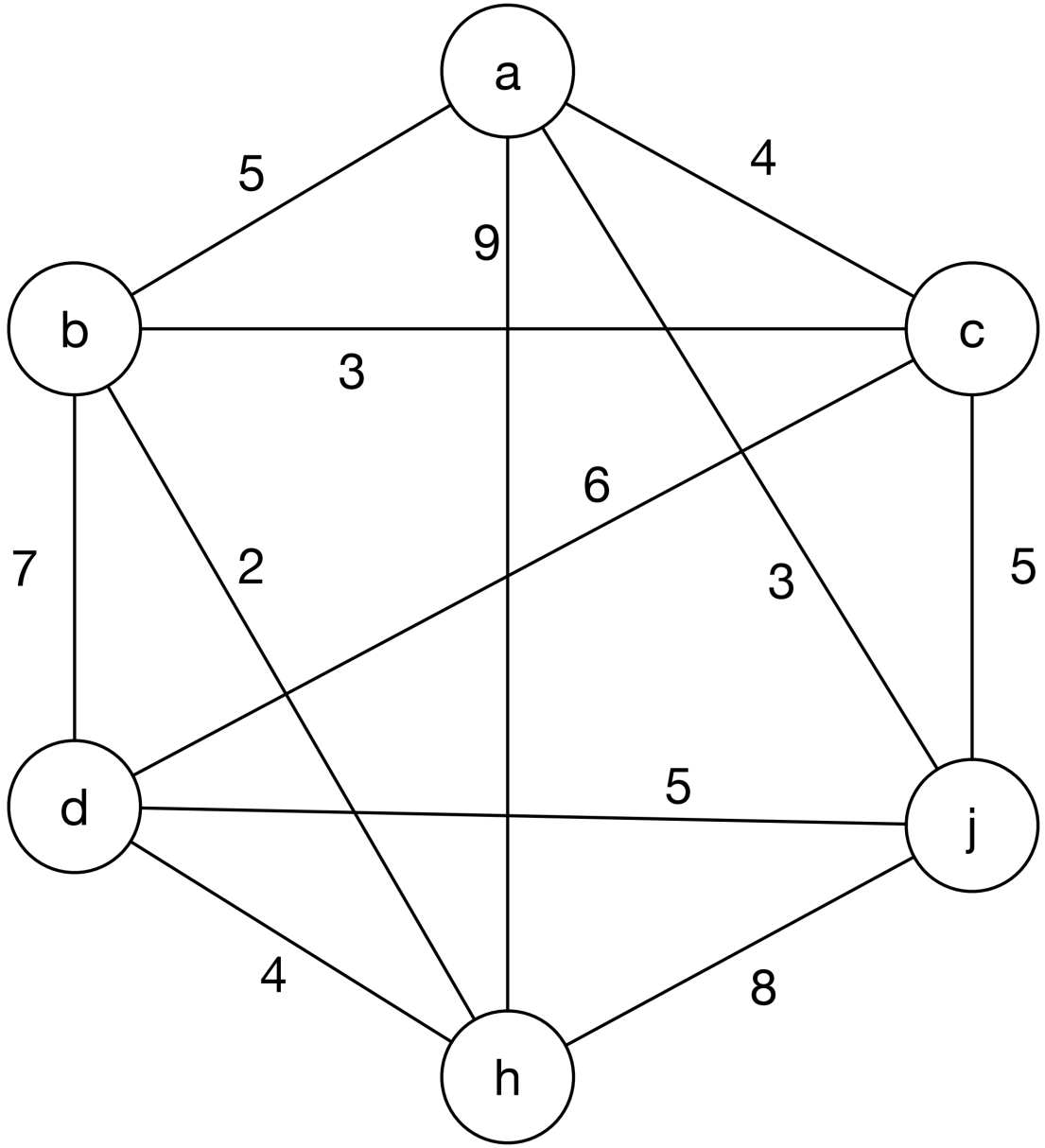
E = {ab, ac, ah, bc, bd,
bh, cj, cd, dj, dh, jh}

V' = {a, c, j, b, d, h}

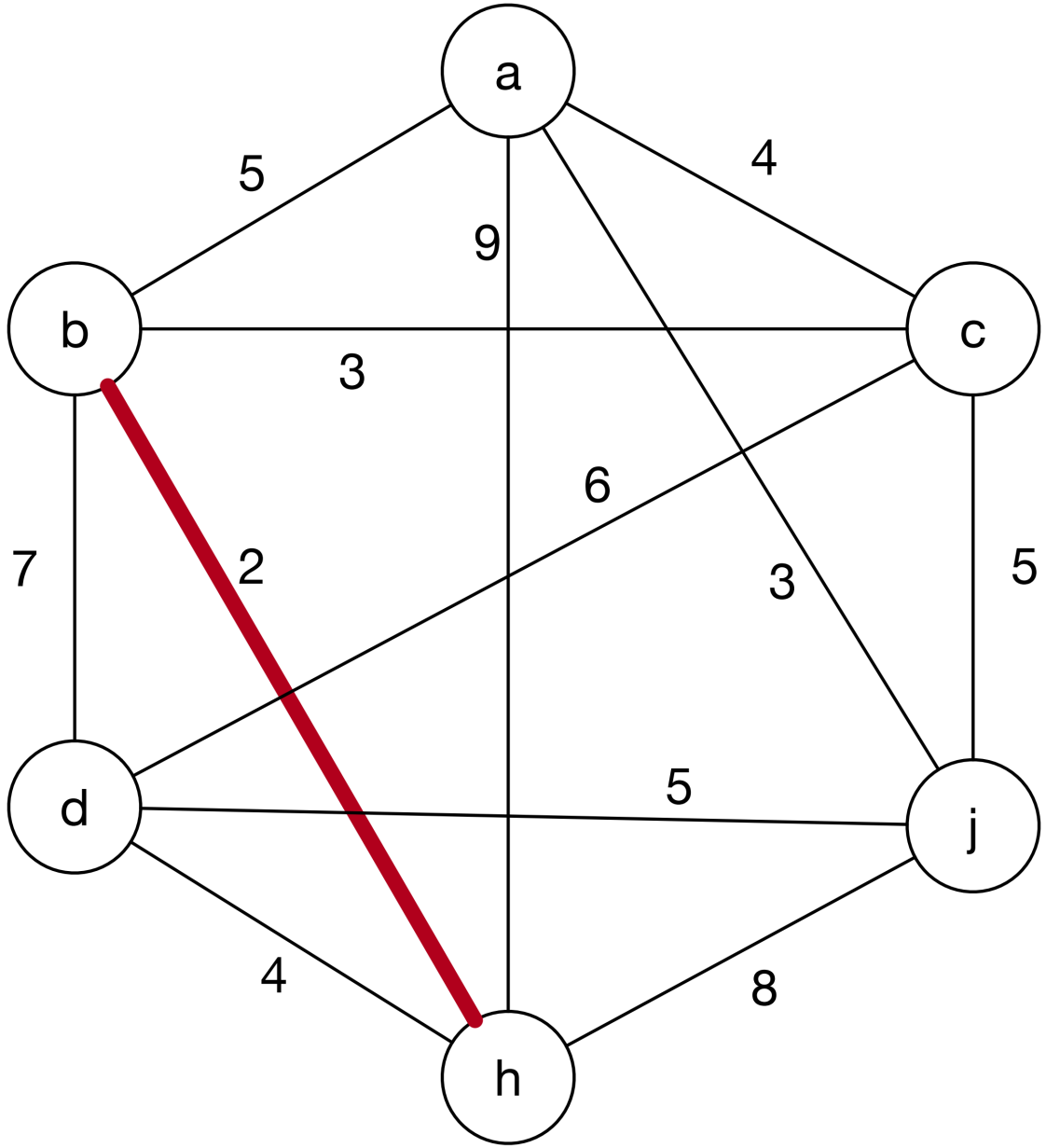
E' = {ac, cj, ab, bd, dh}

Krushkal’s Algorithm

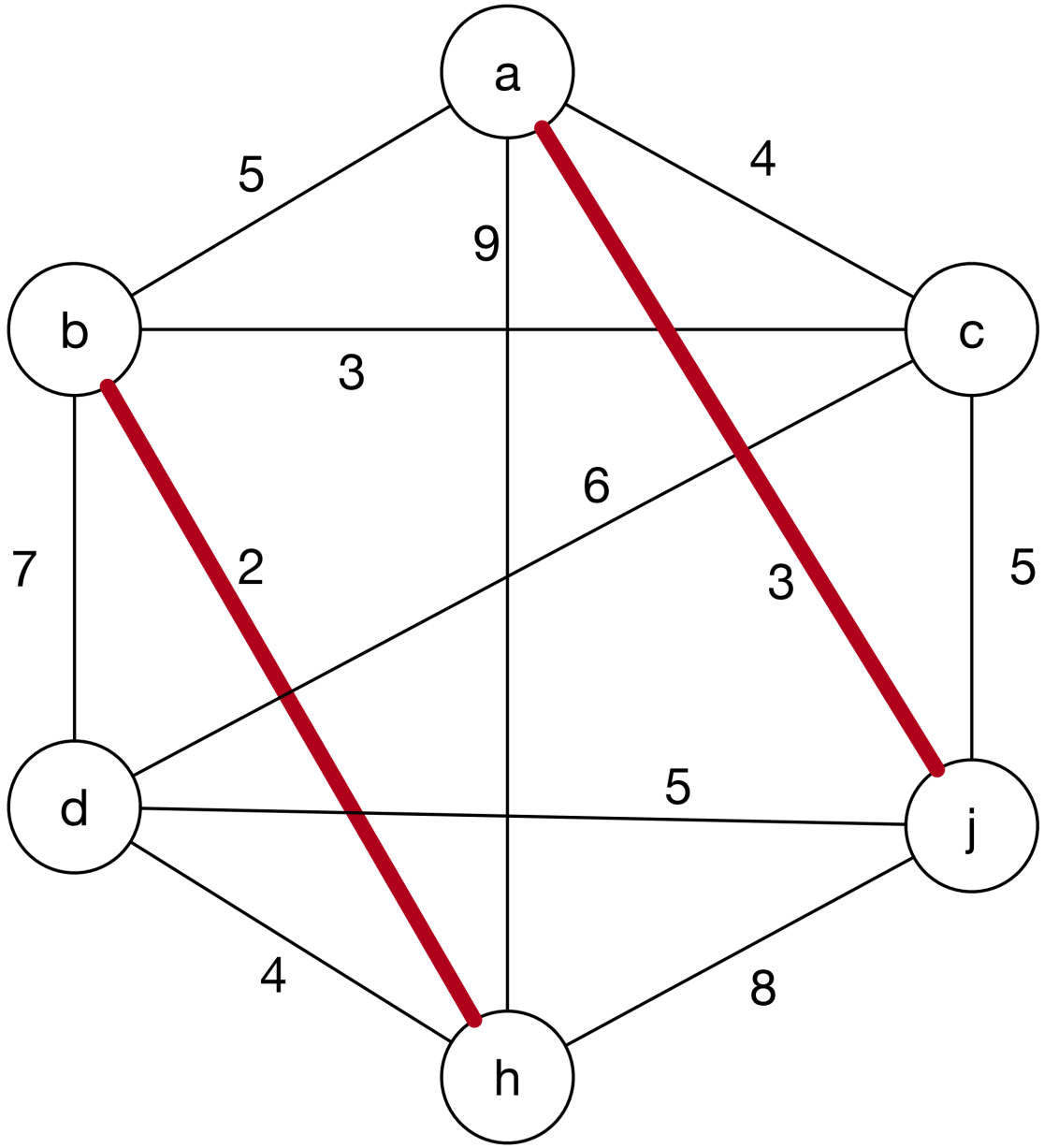
```
function kruskals(G)
        // using disjoint sets, find and union
        let E' = {}
        sort edges in E
        make singleton sets of all nodes in V
        for each edge, (u,v) in E
                if find(u) != find(v)
                        add edge to E’
                        union(u, v)
        return (V, E')
```



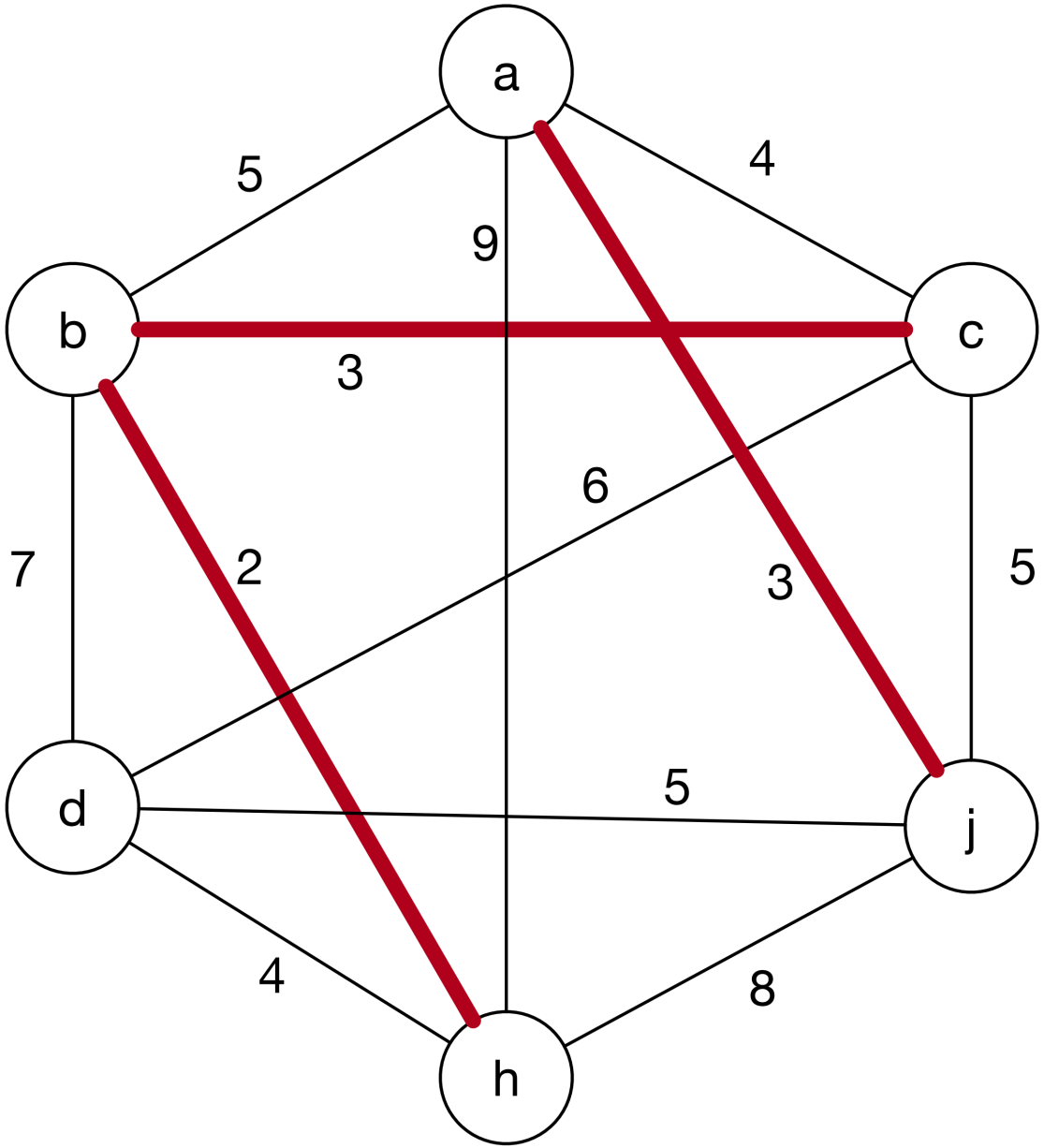
edge	weight
bh	2
aj	3
bc	3
ac	4
dh	4
ab	5
cj	5
dj	5
cd	6
bd	7
jh	8
ah	9



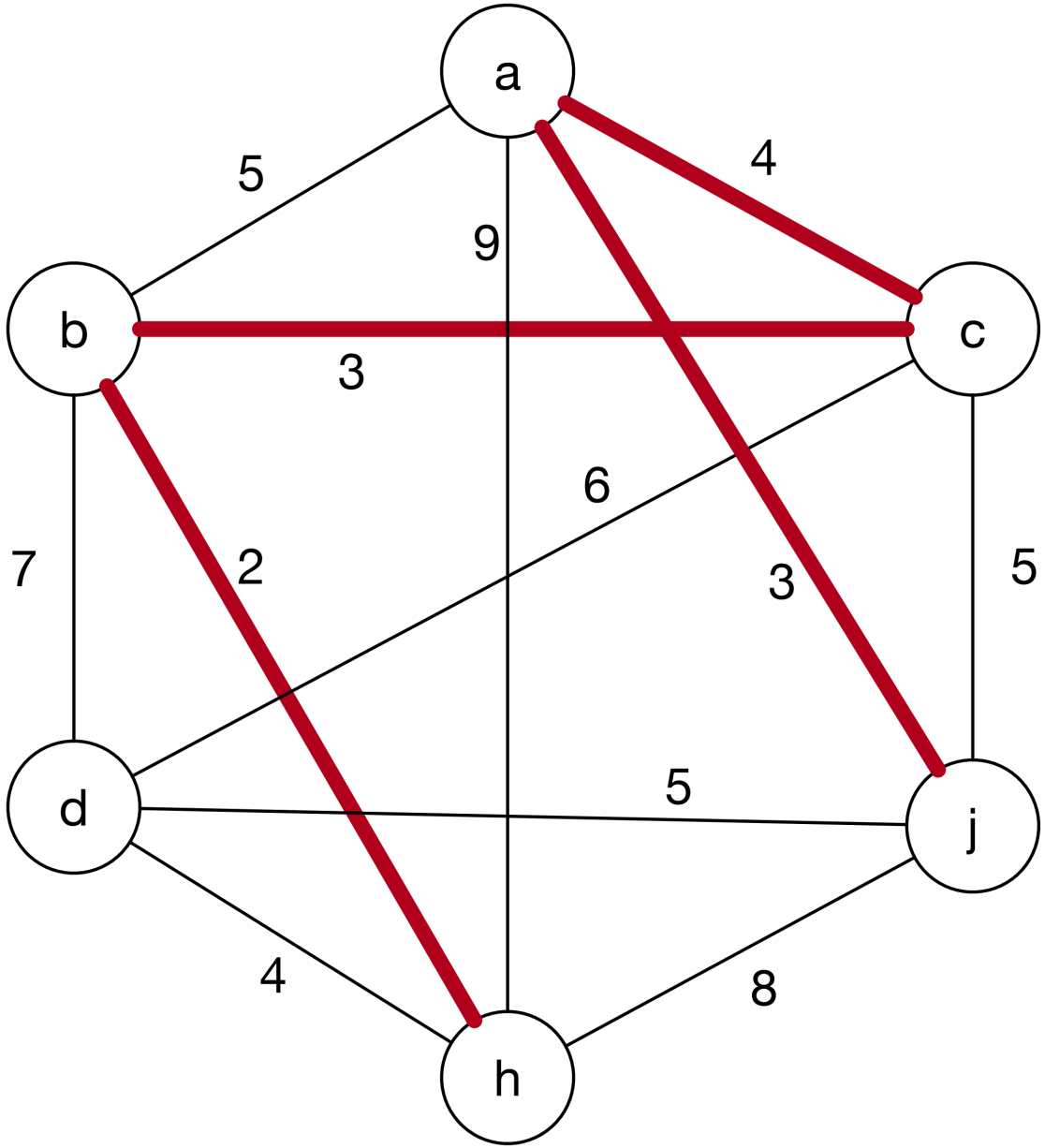
edge	weight
bh	2
aj	3
bc	3
ac	4
dh	4
ab	5
cj	5
dj	5
cd	6
bd	7
jh	8
ah	9



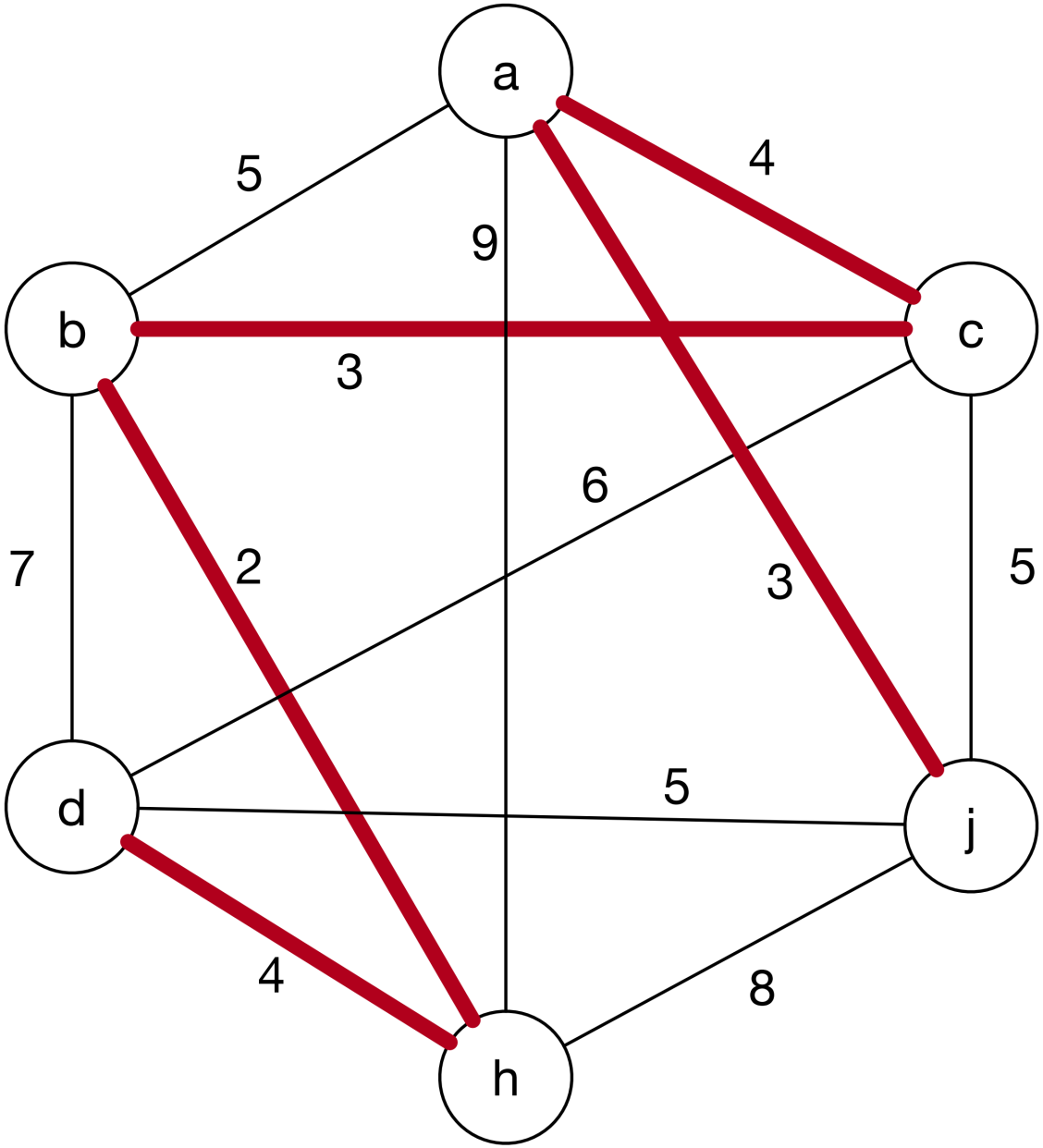
edge	weight
bh	2
aj	3
bc	3
ac	4
dh	4
ab	5
cj	5
dj	5
cd	6
bd	7
jh	8
ah	9



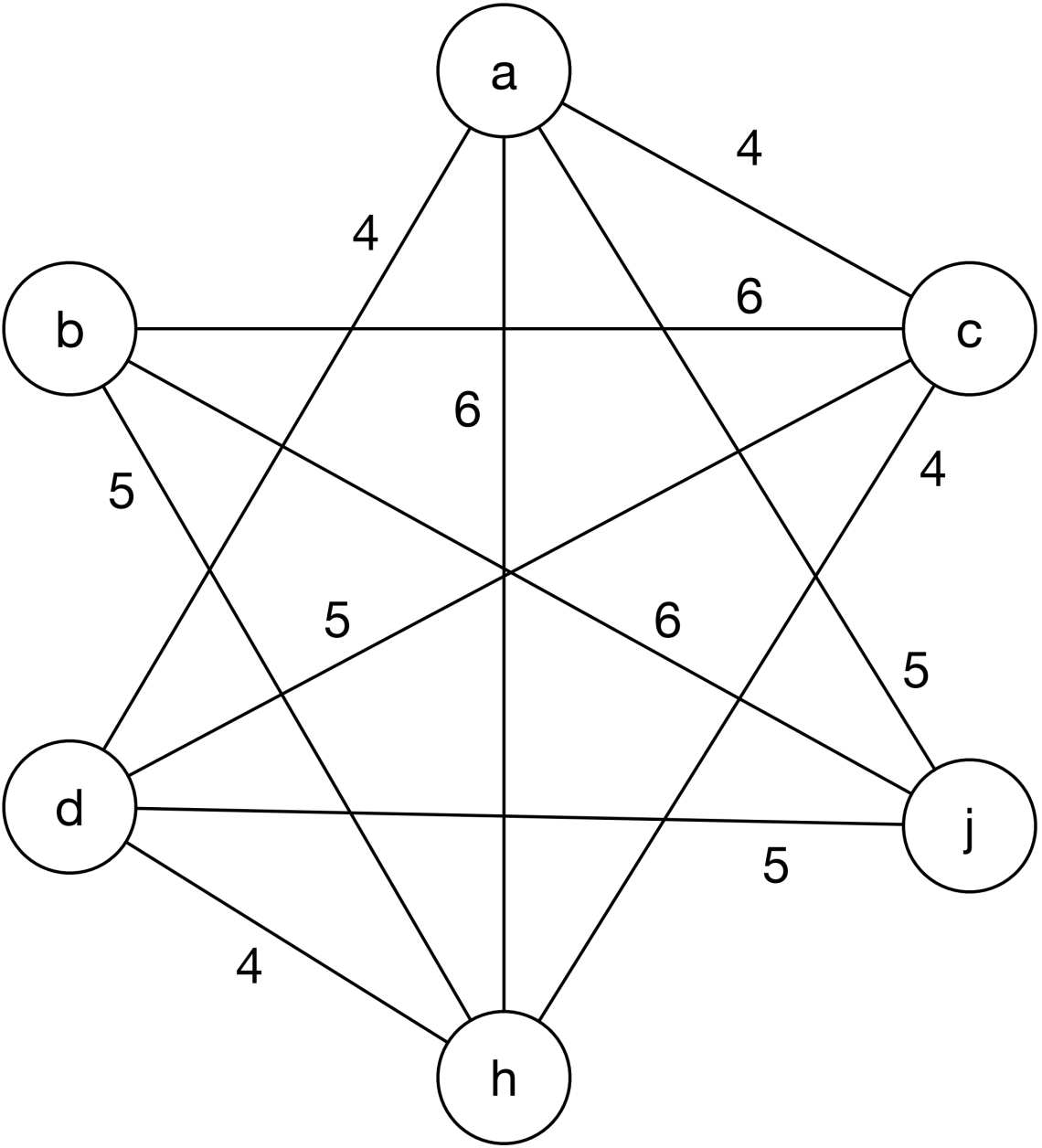
edge	weight
bh	2
aj	3
bc	3
ac	4
dh	4
ab	5
cj	5
dj	5
cd	6
bd	7
jh	8
ah	9



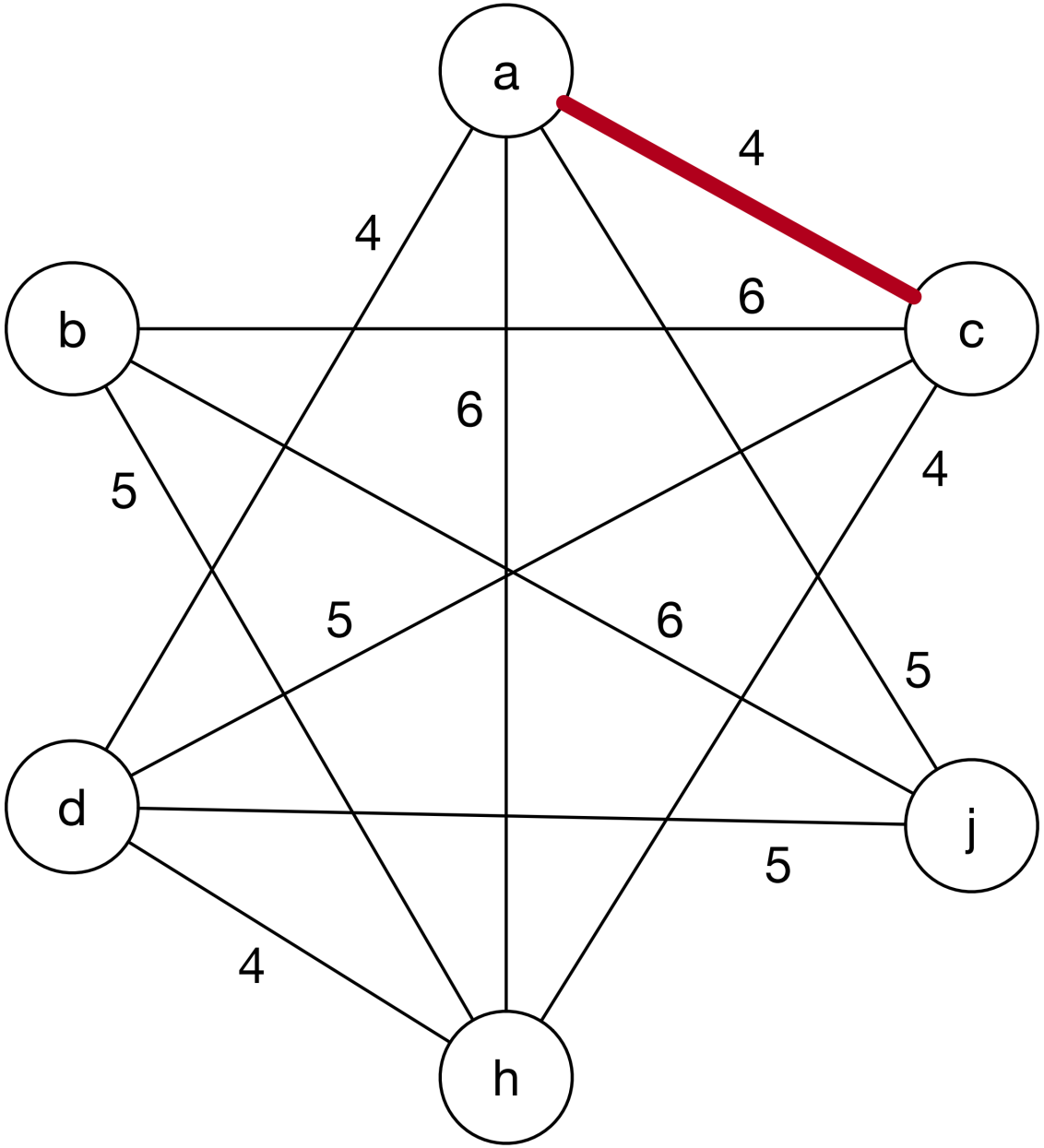
edge	weight
bh	2
aj	3
bc	3
ac	4
dh	4
ab	5
cj	5
dj	5
cd	6
bd	7
jh	8
ah	9



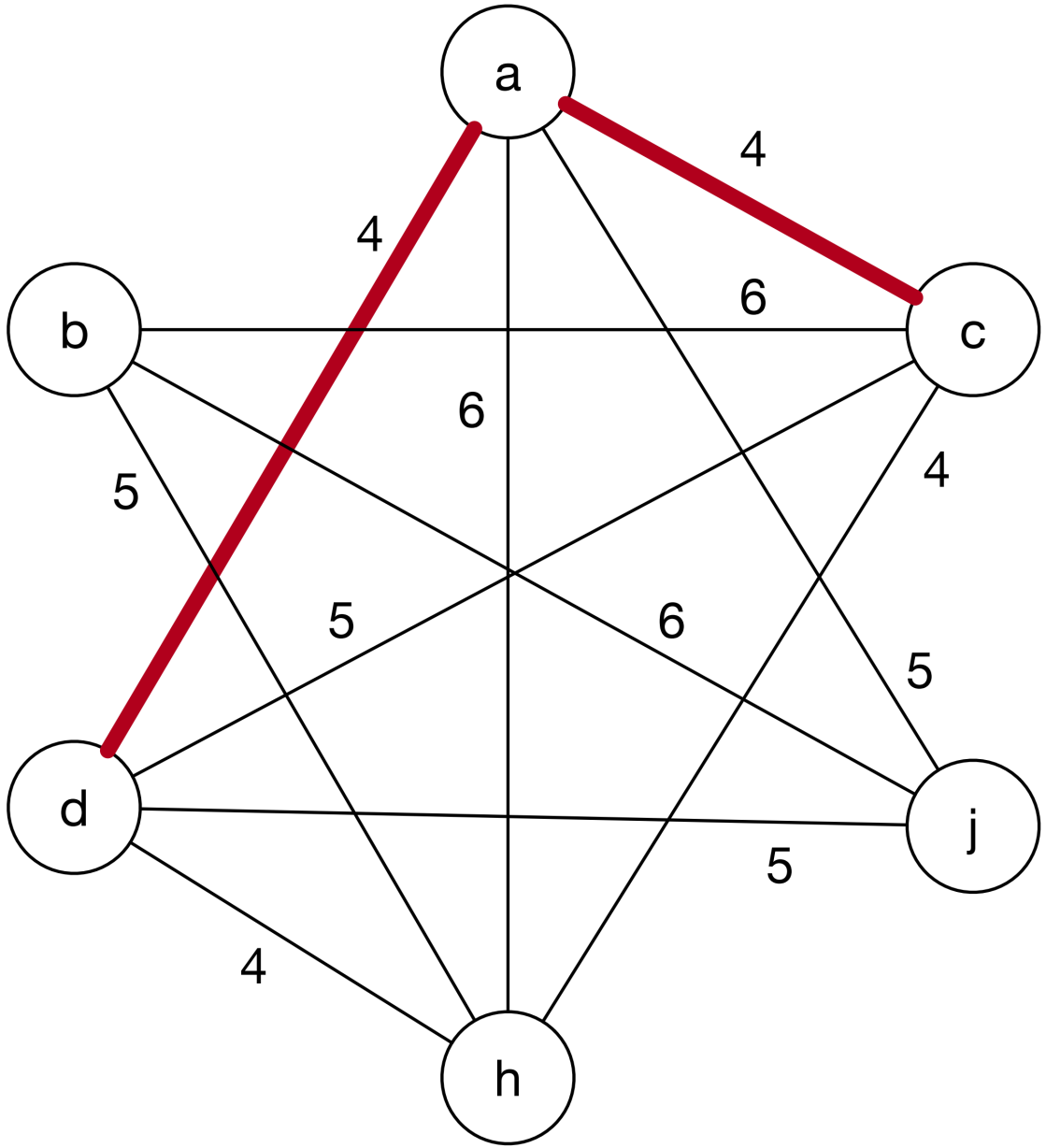
edge	weight
bh	2
aj	3
bc	3
ac	4
dh	4
ab	5
cj	5
dj	5
cd	6
bd	7
jh	8
ah	9



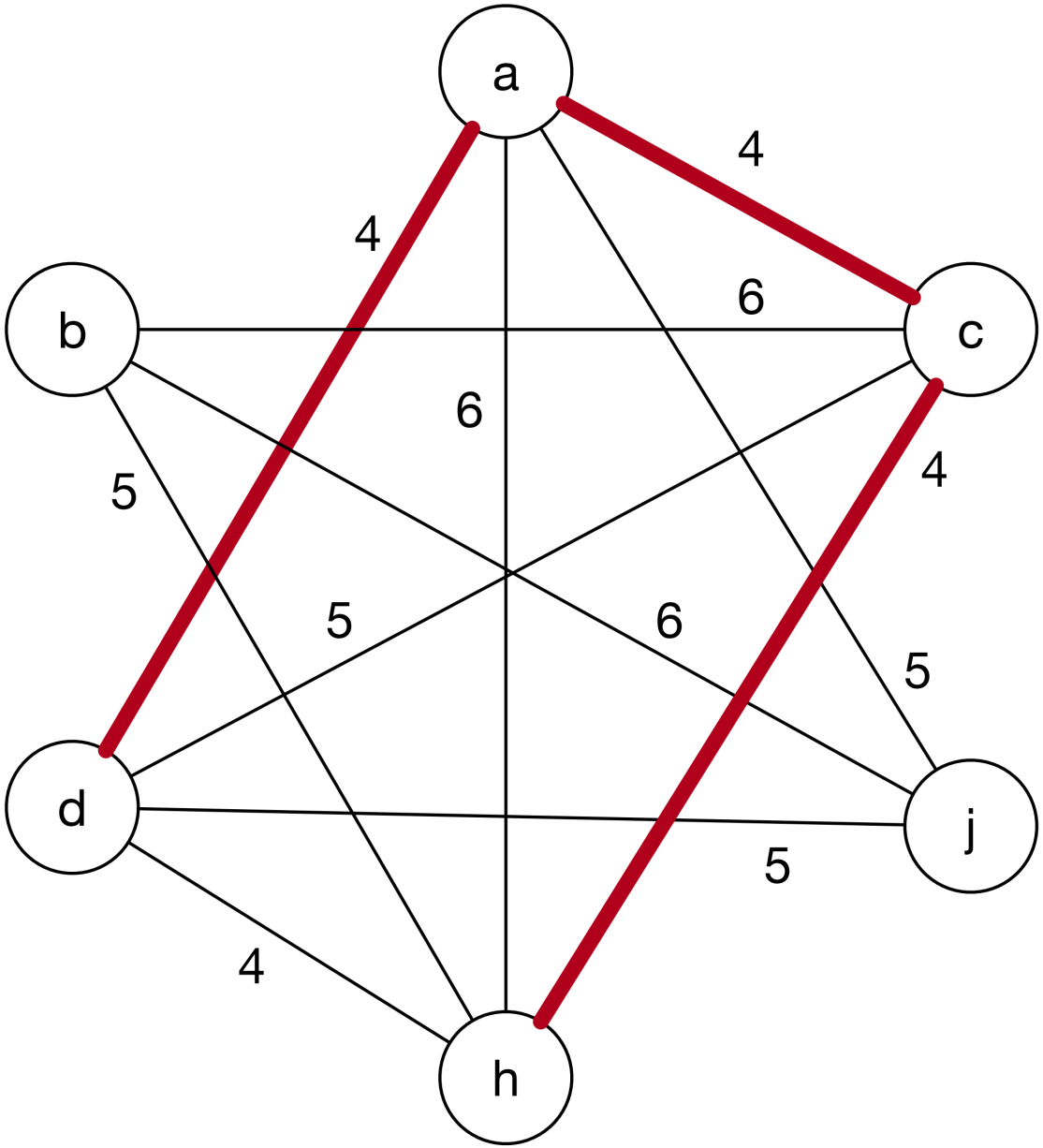
edge	weight
ac	4
ad	4
ch	4
dh	4
aj	5
dj	5
cd	5
bh	5
bc	6
bj	6
ah	6



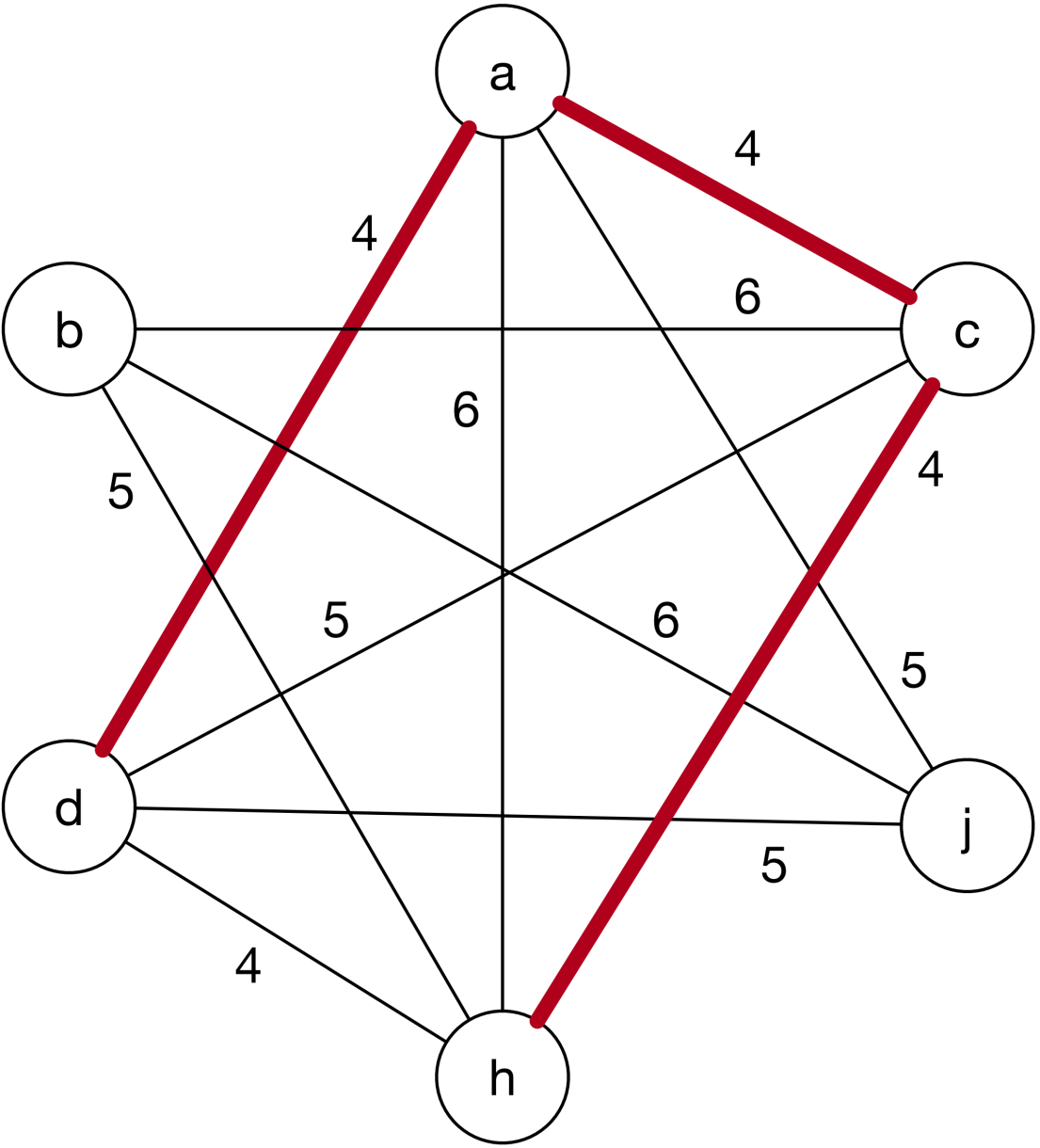
edge	weight
ac	4
ad	4
ch	4
dh	4
aj	5
dj	5
cd	5
bh	5
bc	6
bj	6
ah	6



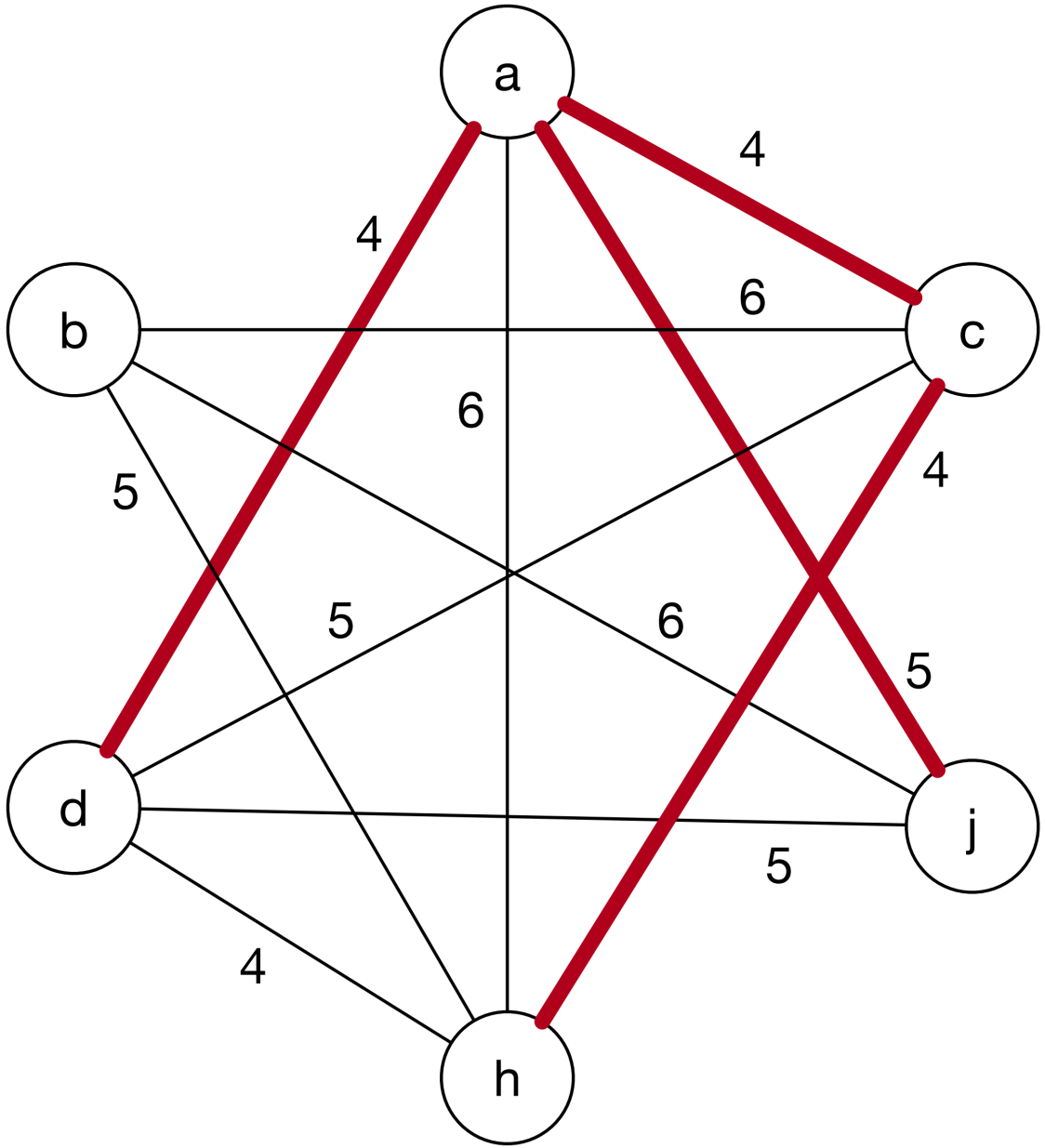
edge	weight
ac	4
ad	4
ch	4
dh	4
aj	5
dj	5
cd	5
bh	5
bc	6
bj	6
ah	6



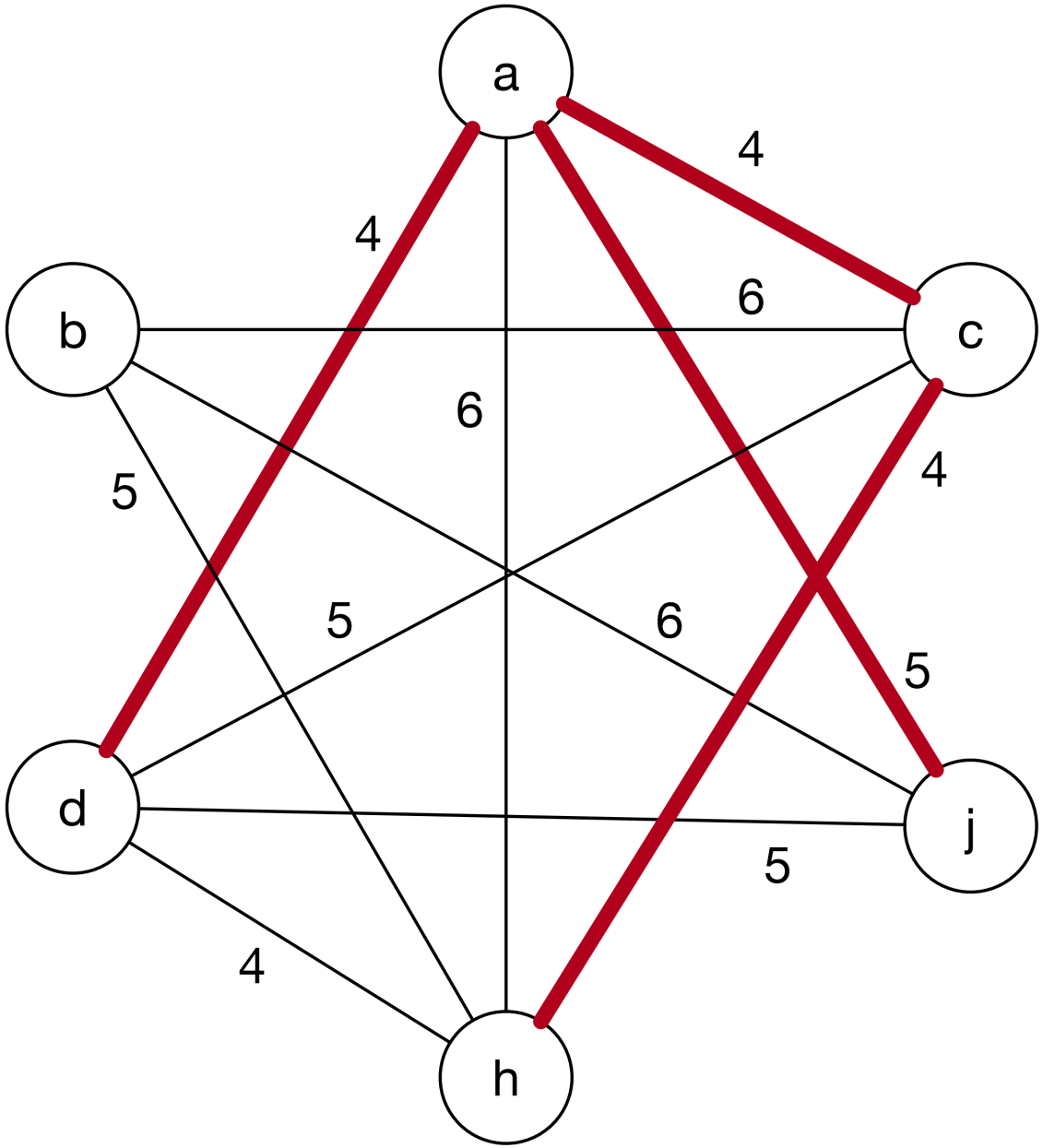
edge	weight
ac	4
ad	4
ch	4
dh	4
aj	5
dj	5
cd	5
bh	5
bc	6
bj	6
ah	6



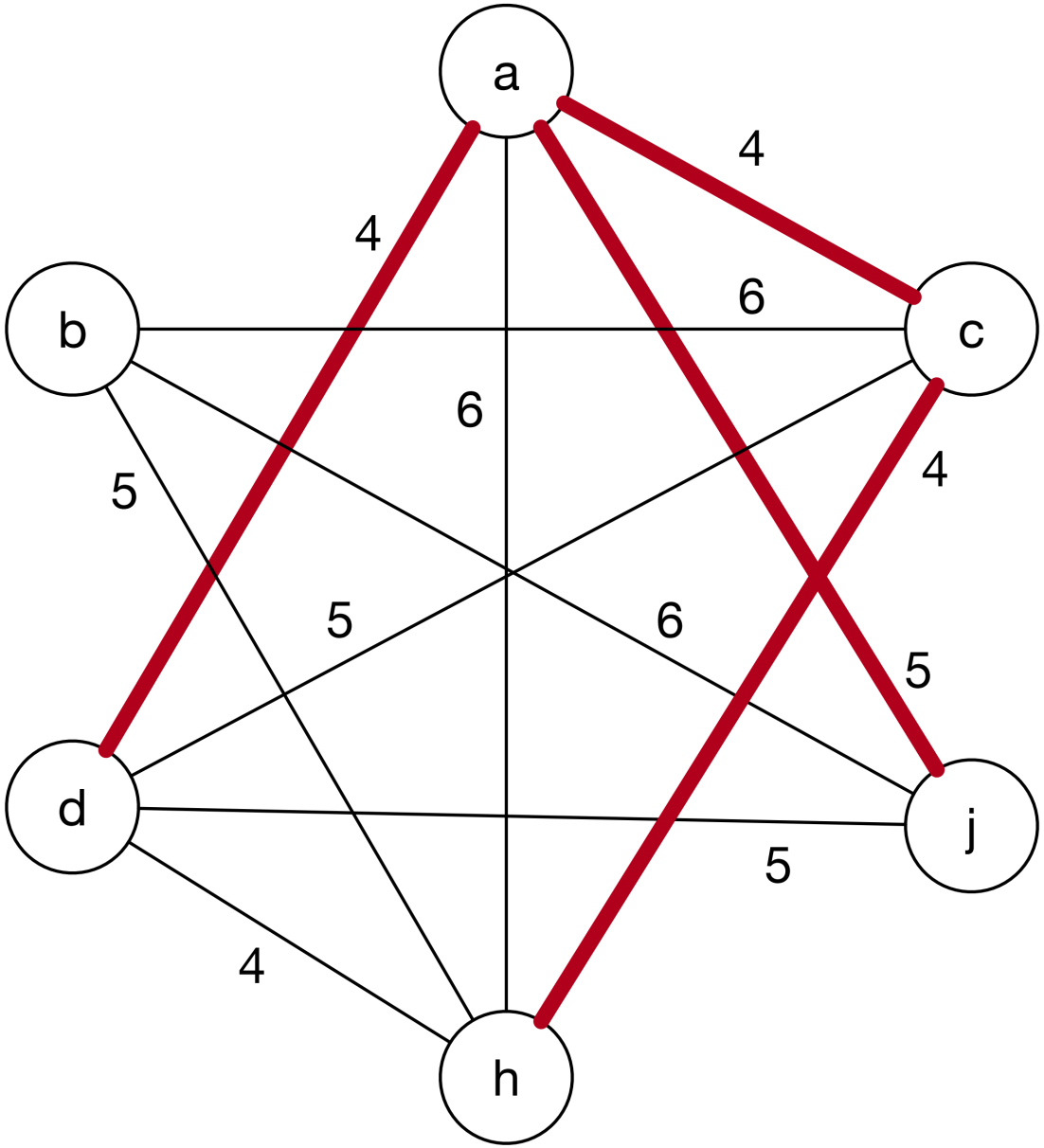
edge	weight
ac	4
ad	4
ch	4
~~dh~~	~~4~~
aj	5
dj	5
cd	5
bh	5
bc	6
bj	6
ah	6



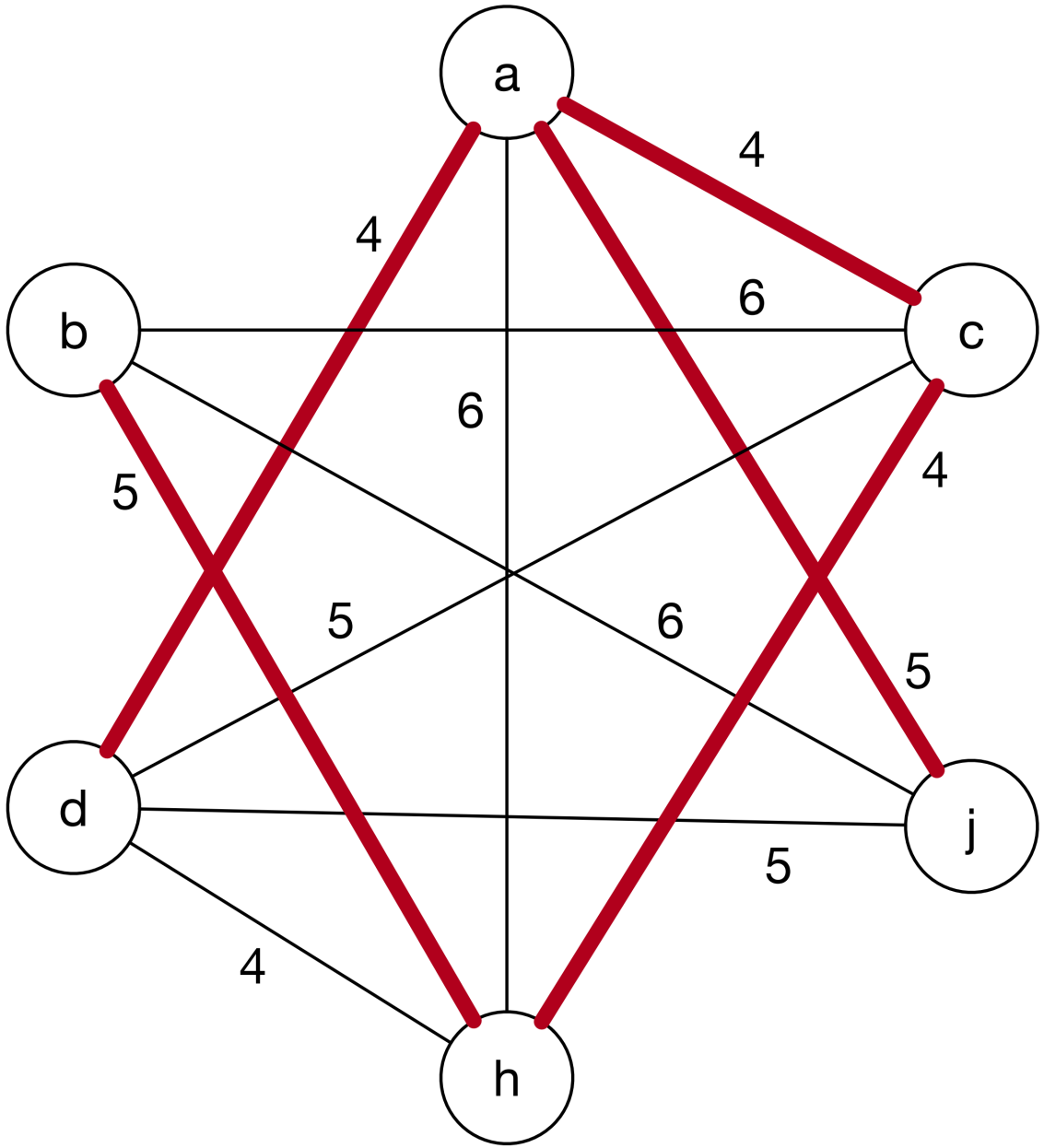
edge	weight
ac	4
ad	4
ch	4
~~dh~~	~~4~~
aj	5
dj	5
cd	5
bh	5
bc	6
bj	6
ah	6



edge	weight
ac	4
ad	4
ch	4
~~dh~~	~~4~~
aj	5
~~dj~~	~~5~~
cd	5
bh	5
bc	6
bj	6
ah	6



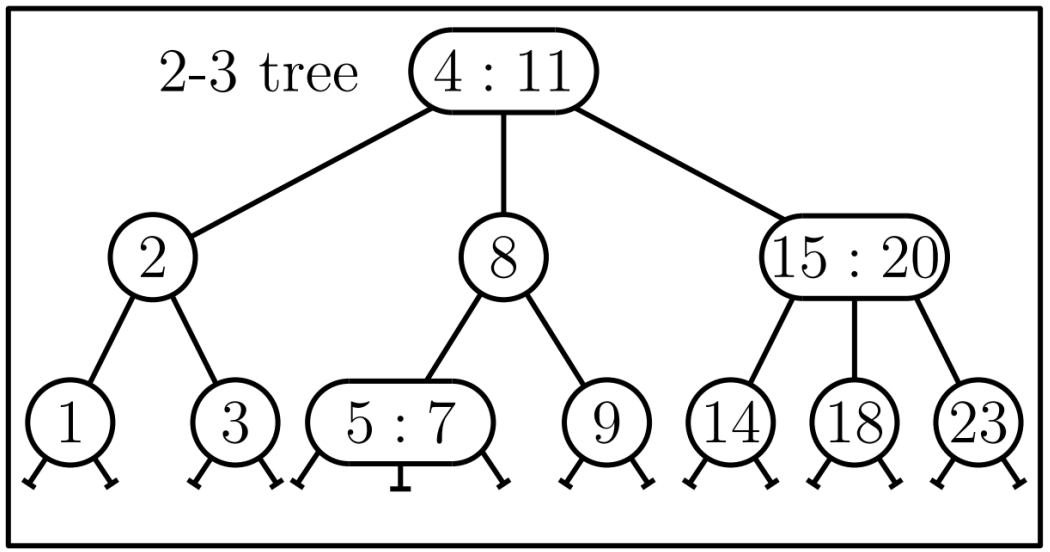
edge	weight
ac	4
ad	4
ch	4
~~dh~~	~~4~~
aj	5
~~dj~~	~~5~~
~~cd~~	~~5~~
bh	5
bc	6
bj	6
ah	6

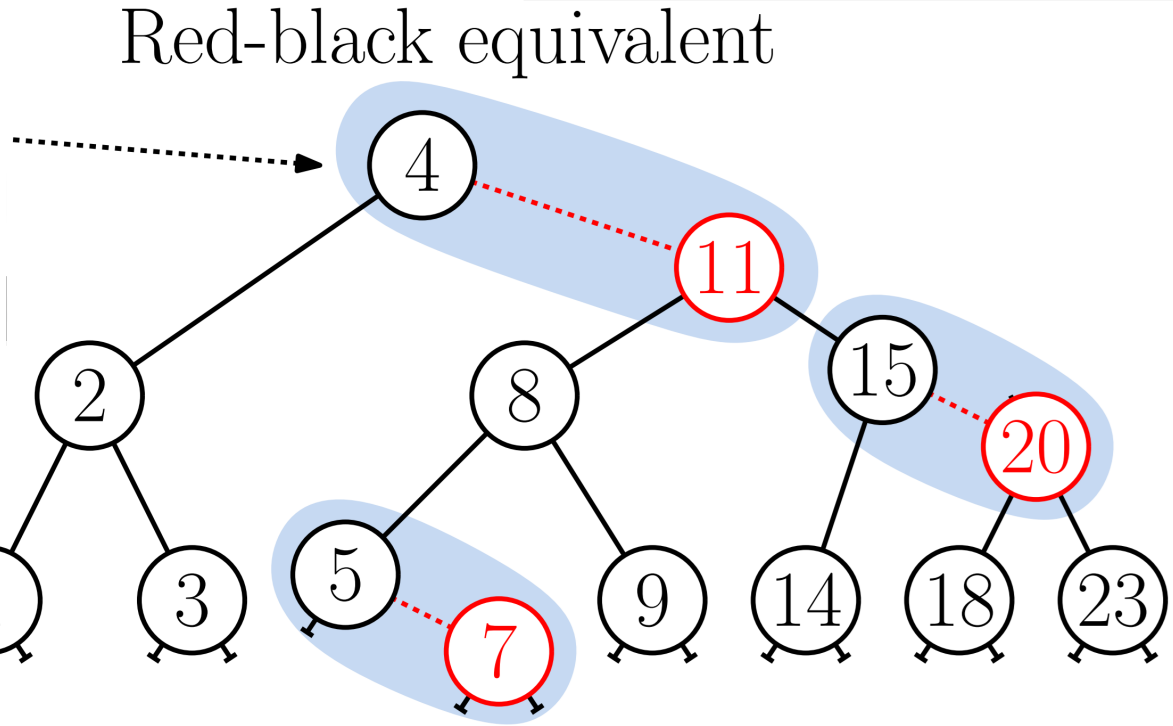


edge	weight
ac	4
ad	4
ch	4
~~dh~~	~~4~~
aj	5
~~dj~~	~~5~~
~~cd~~	~~5~~
bh	5
bc	6
bj	6
ah	6

Red-Black Tree

```
struct t_red_black_node {
    enum { red, black } colour;
    void *item;
    struct t_red_black_node *left,
                *right,
                *parent;
    }
```





Red-Black Tree

- We will label each of “second nodes” of the 3-nodes as red and label all the other nodes as black, we obtain a binary tree with both red and black nodes. The resulting binary tree satisfies the following red-black tree properties:

- Each node is either red or black.
- The root is black. (Since it either arises as a 2-node or the top node of a 3-node.)
- All null pointers are treated as if they point to black nodes (a conceptual convenience).
- If a node is red, then both its children are black. (Since the children of a 3-node are either 2-nodes or the top of another 3-node pair.)
- Every path from a given node to any of its null descendants contains the same number of black nodes. (Since all leaf nodes in a 2-3 tree are at the same depth.)